



DEGREE PROJECT IN MEDIA TECHNOLOGY,
SECOND CYCLE, 30 CREDITS
STOCKHOLM, SWEDEN 2021

Efficient generation and rendering of tube geometry in Unreal Engine

Utilizing compute shaders for 3D line generation

PLATON WOXLER

Efficient generation and rendering of tube geometry in Unreal Engine

Utilizing compute shaders for 3D line generation

PLATON WOXLER

Degree Programme in Media Technology

Date: June 30, 2021

Supervisors: Mario Romero, Patric Ljung

Examiner: Gerald Q. Maguire Jr.

School of Electrical Engineering and Computer Science

Host company: Visualiseringscenter C

Swedish title: Effektiv generering och rendering av tubgeometri i Unreal Engine

Swedish subtitle: Generering av 3D-linjer med compute shaders

Abstract

Massive graph visualization in an immersive environment, such as virtual reality (VR) or Augmented Reality (AR), has the possibility to improve users' understanding when exploring data in new ways. To make the most of a visualization, such as this, requires interactive components that are fast enough to accommodate interactivity. By rendering the edges of the graph as shaded lines that imitate three-dimensional (3D) lines or tubes, one can circumvent technical limitations. This method works well enough when using traditional two-dimensional (2D) monitors, but representing tubes as flat lines in a virtual environment (VE) makes for a less immersive user experience as opposed to visualizing true 3D geometry. In order to accommodate for these requirements *i.e.*, speed and visual fidelity, we need a time efficient way of producing tubular meshes. This thesis project explores how one can generate tubular geometry utilizing compute shaders in the modern game engine, Unreal Engine (UE). Exploiting the parallel computing power of the graphical processing unit (GPU) we use compute shaders to generate a tubular mesh following a predetermined path. The result from the project is an open source plugin for UE, able to generate tubular geometry at rapid rates. While not giving any major advantages when generating smaller models, comparing it to a sequential implementation, the compute shader implementation create and render models $> 40\times$ faster when generating 10^6 tube segments. A secondary effect of generating most of the data on the GPU, is that we avoid bottlenecks that can occur when surpassing the bandwidth of the central processing unit (CPU) to GPU data transfer. Using this tool researches can more easily explore information visualization in a VE. Furthermore, this thesis promotes extended development of mesh generation, using compute shaders in UE.

Keywords

Computer Graphics, Mesh Models, General-Purpose Computing on Graphics Processing Units (GPGPU)

Sammanfattning

Att visualisera stora grafer i en immersiv miljö, såsom VR eller AR, kan förbättra en användares förståelse när de utforskar data på nya sätt. För att få ut det mesta av denna typen av visualiseringar krävs interaktiva komponenter som är tillräckligt snabba för att tillgodose interaktivitet. Genom att visa de linjer, som binder samman en grafs noder, som plana linjer som imiterar 3D-linjer eller rör, kan man undvika att slå i det tak som tekniska begränsningar medför. Denna metoden är acceptabel vid användning av traditionella 2D-skärmar, men att representera rör som plana linjer i VE ger en mindre immersiv användarupplevelse, i kontrast till att visualisera sann 3D -geometri. För att tillgodose dessa krav dvs, tidseffektivitet och visuella kvaliteter, behöver vi ett effektivt sätt att producera 3D-linjer. Denna uppsats undersöker hur man kan generera rörformad geometri med hjälp av compute shaders i den moderna spelmotorn Unreal Engine (UE). Genom att använda compute shaders kan vi utnyttja den parallella beräkningskraften hos en GPU, kan vi generera ett rörformat mesh som följer en förutbestämd bana. Resultatet från projektet är ett open source-plugin för UE, som kan generera rörformad geometri i höga hastigheter. Även om det inte kan visas ge några större fördelar när man genererar mindre modeller, jämfört med en sekventiell implementering, skapar och renderar implementeringen av compute Shaders modeller $> 40\times$ snabbare, när de genererar 10^6 rörsegment. I och med att den större delen av datan skapas på GPU kan vi också undvika den flaskhals som kan uppstå när vi överskrider bandbredden mellan CPU och GPU. Med hjälp av verktyget som skapats i samband med denna uppsats kan människor lättare utforska informationsvisualisering i VE. Dessutom främjar denna uppsats utökad utveckling av mesh-generering med hjälp av compute shaders i UE.

Nyckelord

Datorgrafik, Mesh-modeller, Beräkningar på Grafikprocessorer för Generella Användningsområden

Acknowledgments

I would like to thank my supervisor at Visualiseringscenter C, Patric Ljung, for having proposed the project and also helping me with development and technical support. I would also like to thank my supervisor at KTH, Mario Romero, for giving me valuable feedback on the thesis itself, as well as helping me through the process of conducting a Master's thesis project. Lastly, I would like to thank my examiner Gerald Q. Maguire Jr for assisting me with academic writing and the structure of my $\text{\LaTeX}$ document.

Stockholm, June 2021

Platon Woxler

Contents

1	Introduction	1
1.1	Background	1
1.2	Problem	3
1.2.1	The research problem	3
1.2.2	Scientific and engineering issues	3
1.3	Purpose	3
1.4	Goals	4
1.5	Research Methodology	4
1.6	Delimitations	5
1.7	Structure of the thesis	5
2	Background	7
2.1	Graphics Prerequisites	7
2.1.1	Graphics Pipeline	7
2.1.2	Tessellation stage	9
2.1.3	Forward and Deferred Rendering	10
2.1.4	Raytracing	11
2.1.5	Line Rendering	12
2.1.5.1	Parametric curves and surfaces	12
2.1.5.2	Cylinder representation	13
2.1.5.3	Flat line representation	14
2.2	General-purpose computing on graphics processing units	15
2.2.1	Hardware Architecture	15
2.2.2	Compute Shaders	17
2.3	Unreal Engine	20
2.3.1	Rendering	20
2.3.2	Actors	21
2.3.3	Blueprint Visual Scripting	21
2.4	Related work	22

2.4.1	Virtual Tubelets and Impostors	22
2.4.2	Raytracing tubes	23
2.4.3	Tessellation of parametric surfaces	24
2.4.4	GPU Acceleration in visualization	25
2.4.5	Graph visualization	26
2.4.6	Summary of Graph visualization	26
2.5	Summary	26
3	Method	29
3.1	Research Process	29
3.2	Experiment Design	30
3.2.1	Experimental Setup	31
3.2.2	Experimental Parameters	31
3.2.3	Experimental Data	32
3.2.4	Collected Data	35
3.2.5	Hardware and Software	35
3.3	Method and Data Validity and Reliability	36
4	Development	37
4.1	Compute Shader Implementation	37
4.1.1	Shader Structure Overview	37
4.1.2	Implementation	38
4.1.2.1	Compute Shader first implementation	38
4.1.2.2	Improved compute shader	41
4.1.2.3	Declaration	43
4.1.2.4	Invocation	44
4.2	CPU Implementation	44
4.2.1	Run Time Mesh Component	44
4.2.2	Algorithm	45
5	Results and Analysis	47
5.1	Major results	47
5.2	Statistical validation	55
5.3	Analysis	56
5.3.1	Comparison of sequential and compute shader imple- mentation	56
5.3.2	Sequential implementation	58
5.3.3	Compute shader implementation	59

6	Conclusions and Future work	61
6.1	Conclusions	61
6.2	Limitations	62
6.3	Future work	63
6.3.1	What has been left undone?	63
6.3.1.1	Optimization	63
6.3.1.2	Tool features	64
6.3.2	Next obvious things to be done	65
6.4	Reflections	65
	References	67
A	Source code	75
A.1	Compute Shader	75
A.2	Sequential Implementation	82

List of Figures

2.1	Basic graphics pipeline.	8
2.2	Visual representation of the rasterization process.	9
2.3	Data flow in tessellation stage of graphics pipeline.	9
2.4	Deferred rendering pipeline.	10
2.5	Raytracing example.	12
2.6	Cubic Bézier curve.	13
2.7	Generalized cylinder.	14
2.8	Simplification of Parallel and Sequential execution.	16
2.9	Basic GPU architecture.	17
2.10	Compute Unified Device Architecture (CUDA) thread hierarchy.	18
2.11	The path of a draw in UE.	21
2.12	Example of Blueprint.	22
2.13	Flow chart of a GPU-centric design.	25
3.1	Flow Diagram of the research process for this thesis.	30
3.2	Overview of experimental design.	31
3.3	Spiral generated using 500 centroids.	34
4.1	Compute shader overview in UE.	38
4.2	Generating a tube segment with three vertices representing the tube segment.	39
4.3	A vertex P is positioned on the circle of a radius r around a center vertex.	41
4.4	Visual artefact of tube segments meeting.	41
4.5	Meeting tube segments without visual artefact.	43
5.1	Render time results of compute shader implementation.	52
5.2	Render time results of sequential implementation.	53
5.3	Presenting execution time of sequential and compute shader implementation when using different level of detail (LOD).	54

List of Tables

2.1	Comparison of related line rendering techniques and my own .	27
3.1	Hardware specifications	35
5.1	Average results from experiment with $LOD = \{3, 9\}$	48
5.2	Presenting average results comparing render time of a frame when generating N segments with $LOD = \{3, 9\}$ on CPU and GPU	49
5.3	Presenting overview of average results when recording 100 frames using the compute shader implementation	50
5.4	Presenting the summary of results from running the sequential implementation	51
5.5	Results from Mann-Whitney U tests performed on the results from executing the CPU and GPU implementations of the algorithm	55
5.6	Results from Mann-Whitney U tests performed on the results obtained when running the same algorithms with varying LOD .	56
5.7	Presenting the results from dividing the frame rendering time of the sequential implementation with the compute shader implementation	58
5.8	FDynamicMeshVertex and Triangle data structures and their size	60

Listings

2.1 Pseudo code of GPU and CPU array addition	19
---	----

List of acronyms and abbreviations

2D	two-dimensional
3D	three-dimensional
API	Application Programming Interface
AR	Augmented Reality
CPU	central processing unit
CUDA	Compute Unified Device Architecture
dCache	direct cache
DRAM	Dynamic Random Access Memory
fps	frames per second
G-buffer	geometric buffer
GPGPU	General-purpose computing on graphics processing units
GPU	graphical processing unit
GST	geometry shader tessellation
HLSL	High Level Scripting Language
LOD	level of detail
PD	primitive distributors
RHI	Render Hardware Interface
RMC	run time mesh component
SIMD	Single Instruction Multiple Data
SIMT	Single Instruction Multiple Threads
SM	Streaming Multiprocessors
SOL	speed of light
SSAO	screen space ambient occlusion
UE	Unreal Engine
UE5	Unreal Engine 5

usf	Unreal Engine shader file
VE	virtual environment
VR	virtual reality
VRAM	Video Random Access Memory
VST	vertex shader tessellation

Chapter 1

Introduction

This chapter begins by giving a general background for the problem (see Section 1.1), describes the specific problem that this thesis addresses (Section 1.2), and the purpose and goals of this thesis project (Sections 1.3 and 1.4). Section 1.5 describes the research methodology that has been adopted. Section 1.6 enumerates the delimitations of this thesis, including what it explicitly excludes. Finally, Section 1.7 outlines the structure of the thesis.

1.1 Background

In this thesis, the term *information visualizations* refers to the visualizations of abstract data that does not have a real world connection to physical space [1]. This could include stock market fluctuation, airplane traffic, or social connections between persons on a digital platform. Visualizing data in an immersive environment using VR tools allows the user to explore the data in new ways and gain deeper and different insights faster as opposed to using traditional methods such as a 2D representation on a monitor [2, 3]. O. Kwon, *et al.*, showed that immersive 3D visualization for graphs works especially well when we consider bigger and more complex graphs when we judge them based on interaction speed and accuracy [2]. The speed and accuracy in this case refers to how fast and accurate a user can solve a specific problem related to the graph.

When designing for immersive visualization, there are a few things that are required of a system. First of all, it must be fast. The system is recommended to run at, at least, 60 frames per second (fps) to be comfortable for an extended time of use [4]. It should also respond to user input within 100 ms in order to

induce a proper sense of immersion. It also needs to display its content with a certain level of fidelity, since the goal is, as stated by Bryson: “to create the effect of interacting with things, not with pictures of things” [5].

For the case of interactive graph visualization in VR, we can draw a few conclusions by utilizing these previously mentioned principles (*i.e.*, immersive 3D, 60 fps, maximum response time of 100 ms, and high fidelity). First, it is not enough to simply render connections between nodes as planes, since this would disrupt the illusion of a truly three dimensional world. Therefore, these connections should be three-dimensional. One way of generating these connections is to use a spline. A spline is defined by piecewise polynomials and has a continuous first derivative [6]. Splines are often used to generate lines and once a line has been created you can fit a mesh around it to create different shapes such as tubes, roads, *etc.* Thus, splines are useful for creating connections and rendering 3D lines between nodes in a graph visualization. Second, we must also generate and render the connections at sufficiently fast rates to achieve the desired frame rates and response times so as not to disrupt the user’s sense of immersion. One way of achieving this is to accelerate the line generation by parallelizing the line generation algorithm and running it asynchronously on several threads in the CPU or by utilizing the parallel computing power of a GPU [7, 8].

This project aims to create a tool for generating 3D lines (tubes) from splines using the game engine called the Unreal Engine (UE) [9]. This tool will take advantage of *compute shaders* to utilize the GPU. A shader is a program that runs on the GPU. Shaders have in the past mostly been used for rendering graphics. In contrast, compute shaders use the same resources as conventional shaders but can be used for arbitrary calculations, this is also known as General-purpose computing on graphics processing units (GPGPU). The benefit of using a GPU to do processing is its increased processing power, memory bandwidth, and parallelism [10]. Earlier limitations of General-purpose computing on graphics processing units (GPGPU) were limited GPU memory and slow transfer speeds between the CPU and the GPU. With the rise in interest of GPGPU, the hardware has rapidly developed and many of the earlier problems have been dealt with. This tool’s primary use will be interactive massive graph visualization* in an immersive environment. This project is in collaboration with Visualiseringscenter C [11].

* For the purpose of this thesis we will defined massive graphs as graphs with more than 10^5 connections.

1.2 Problem

This thesis project aims to fill the gap due to the lack of efficient 3D tube rendering for information visualization in UE. The research question will be presented in Section 1.2.1. Following this, Section 1.2.2 describes some of the research issues addressed in this thesis.

1.2.1 The research problem

The original problem was proposed by Visualiseringscenter C and is connected to their research in immersive visualization. They want to explore graph visualization in a VE and they need a tool that can render 3D tubes since 2D lines are perceived as flat planes when viewed within a VE, which is undesirable. UE lacks native support for efficient generation of 3D tubes and my task is to create such tool. The research question stems from the original problem and is stated as follows:

What is the performance impact on time efficiency of adapting a cylinder mesh generation algorithm, from running on the CPU to executing on the GPU of a mid-level graphics workstation*?

1.2.2 Scientific and engineering issues

The main scientific issues this thesis project will examine is how to efficiently generate a tubular spline mesh following the trajectory of a cubic Bézier curve. This involves minimizing unnecessary data transfer between CPU and GPU memory as well as efficiently utilizing hardware resources. It also involves reducing unnecessary overhead by minimizing calculations needed for each thread's invocation in the compute shader.

1.3 Purpose

The purpose of this thesis is to find out to what extent we can use consumer products and software, specifically the hardware (as described in Table 3.1 on page 35) and UE as software, to effectively visualize tubes for use in information visualizations using compute shaders. The results from this thesis will be of use for anyone interested in visualizing tubular structures. This could

* We will define a mid-level graphics workstation as an Intel® Core®i7-8750H as the CPU with an NVIDIA GeForce® GTX 1050Ti 4G as the GPU.

includes researchers wanting to visualize graph structures in VR, or anyone wanting to explore data where 3D lines could be of use.

1.4 Goals

The ultimate goal of this project is to create a tool in UE that can draw 3D lines (tubes) with sufficient efficiency that it can be used for scientific data visualizations in immersive environments. This can be split into two subgoals:

- Subgoal 1** Create a tool capable of visualizing 3D tubes for use in information visualization at Visualiseringscenter C and
- Subgoal 2** Implement and evaluate different optimization techniques for GPU and CPU line generation.

1.5 Research Methodology

My aim is to achieve an efficient method to generate tubes following a trajectory represented by a quadratic Bézier curve. The use of the Bézier curve comes naturally since it is one of the most common ways of representing parametric curves in computer graphics [12]. It is also the way that UE represents its spline segments [13], thus making it easier to integrate this tube generation algorithm with the UE interface.

Compute shaders will be used to generate the geometry needed to render the tube structures. Given the spline data using the GPU, these compute shaders will create geometric primitives. Moreover, generating the geometry directly on the GPU will help minimize the costly data transfer between CPU and GPU memory.

Evaluating the proposed method, will require a quantitative analysis of the rendering time of different scenes. These scenes could include real world data, as used in [14] where they measured the fps during a fuel injection visualization. This evaluation will also include synthesized data as in [15, 16]. In the paper by B. Russig, *et al.* [15], they evaluated their method by rotating their datasets around its center for 1000 frames and timing it. They also had two configurations for each dataset (near and far) which could give an indication of how the algorithm handles different fill rates. The data sets will vary in size in order to assess if one algorithm (CPU based) is better than the other (GPU based) at any particular dimensions or configurations of data. Timing in this thesis will occur both of the initial generation and rendering of

the datasets. Knowing the initial rendering time is important as the user might want to change between datasets during use and give an indication on how well the models would behave during interaction.

During development, I tried to balance the computations in the graphic pipeline so as not to experience bottlenecks during rendering. I followed the instructions of Tomas Möller in [12] to analyze the different stages of the rendering pipeline and to ensure a balanced workload throughout.

1.6 Delimitations

Although the tool is meant to be used for data visualization, I will not make any effort to ensure that any implementation looks good or that it can give results that increase the sense of quality of a visualization.

The tool will only be tested using one specific machine. The project does not have access to different machines and thus will only use the machine and software described in Section 3.2.5.

1.7 Structure of the thesis

Chapter 2 presents relevant background information about computer graphics in general as well as related work regarding mesh generation and rendering of tubular geometry. Readers who are knowledgeable in the field of computer graphics can skip Section 2.1 in this chapter. Chapter 3 presents the methodology and method used to solve the problem as well as an overview of the research process for this thesis. For information on how the solution presented in this thesis works, see Chapter 4. The results, their analysis and discussion are presented in Chapter 5. The thesis ends with Chapter 6 where I conclude with the results, discuss improvements and suggest further research building upon this thesis.

Chapter 2

Background

Section 2.1 provides basic background information on graphics and rendering methods. Section 2.2 presents basic features of GPGPU while Section 2.3 covers the specific engine used when building the solution. Section 2.4 describes related work in line rendering techniques and GPU acceleration methods in visualization.

2.1 Graphics Prerequisites

This section is split into four parts: Section 2.1.1 describes the basics of a graphics processing pipeline, Section 2.1.2 describes the tessellation stage present in most modern graphics pipelines, Section 2.1.4 covers the basics of raytracing. Finally, Section 2.1.5 introduces the fundamentals of line rendering. These parts introduce the reader to the necessary concepts and terminology in computer graphics.

2.1.1 Graphics Pipeline

Going from application to visualization on the screen is a process that utilizes the graphics pipeline. The pipeline does not look the same for every GPU manufacturer but usually has the basic components in common. We can divide the process into four stages (shown in Figure 2.1): application, geometry processing, rasterization, and pixel processing. The stages of this pipeline execute in parallel and each stage depends on the output of the previous stage [12]. The various shader stages in the pipeline are programmable processes that are accessible to the developer. These shaders give developers control over how the data in the pipeline is processed.

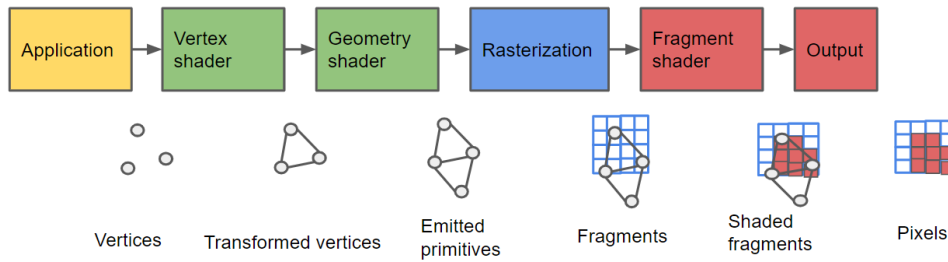


Figure 2.1: Basic graphics pipeline with Application stage (yellow), Geometry processing (green), Rasterization (blue), and Pixel processing (red). This figure was inspired by [12, 17].

The output from the application into the pipeline is geometric primitives which could be points, lines, triangles, or other geometric primitive. These are represented by vertices which hold information on position, color, velocity, and other relevant data for our application [17].

The vertex shader is responsible for two main things [12]. The first is updating the position of a vertex linear transformation to accommodate the user's position or a flow of time. The other purpose is to evaluate other data that the programmer wants to associate with the vertex. This could include the vertex's normal or velocity. The geometry shader has access to information on not only the immediate vertex but also its neighbours. This can be used to create new geometry and do calculations that need more than a single vertex. The primitives are then divided into fragments which translates them from a 3D world into 2D space, this is known as rasterization. Fragments hold information which has been interpolated from the primitive's vertices where the fragment is located. The fragment shader is usually the last step before the images are shown. Here we use the information in the fragments to decide the appearance of the pixel that the fragment occupies, see Figure 2.2.

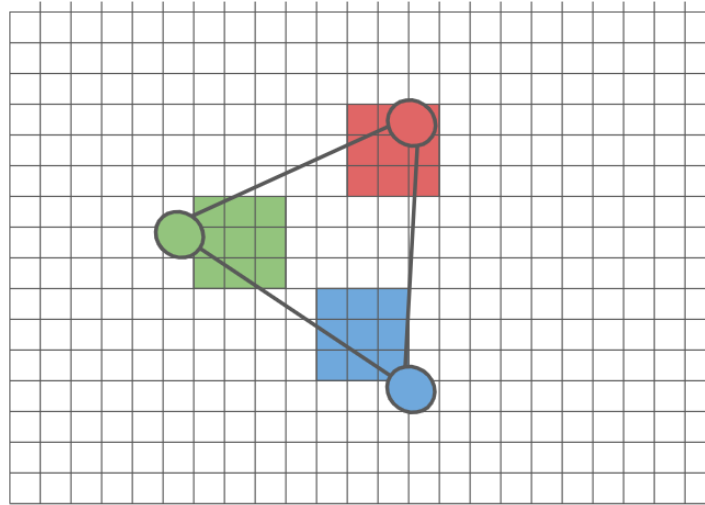


Figure 2.2: Visual representation of the rasterization process.

2.1.2 Tessellation stage

An optional stage present in modern GPUs is the tessellation stage [12]. Tessellation is the process of subdividing a surface into primitives. The stage is adept at processing and rendering curved surfaces such as the parametric surfaces described in Section 2.1.5.1. The tessellation stage has three components (see Figure 2.3) which in DirectX [18] terminology are named: hull shader, tessellator, and domain shader.

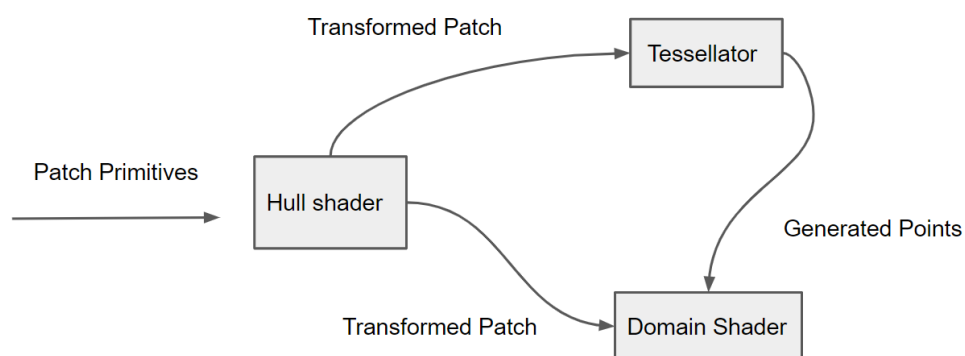


Figure 2.3: Data flow in tessellation stage of graphics pipeline.

The input for the hull shader is a patch primitive comprised of several control points. The patch could be described with, for instance, a Bézier patch

(as described later in Section 2.1.5.1). Each patch is processed by the hull shader and then sent to the tessellator which adds vertices to the patch which increase its level of detail. If no tessellation is required, the processed patch is sent directly to the domain shader. The domain shader operates on each vertex similar to the vertex shader to compute position, normals, and other relevant information. The domain shader outputs triangle primitives to the geometry shader or rasterization stage.

2.1.3 Forward and Deferred Rendering

Section 2.1.1 and 2.1.2 described the rendering pipeline and its varying stages. This section will introduce two popular ways in which data can be handled by the pipeline. *Forward* rendering is possibly the most basic and naive approach and is used to some extent in two of the most popular game engines: Unreal Engine (UE) and Unity [9, 19]. In forward rendering, most light calculations are done in the vertex shader, early in the pipeline.

Deferred rendering, or deferred shading, works slightly different from forward rendering. As the name implies, the process defers lighting calculations until a lighter stage in order to skip unnecessary calculations. The method performs an initial pass over the scene and stores surface information in what is usually known as the geometric buffer (G-buffer) [12] (See Figure 2.4). The G-buffer a collection of buffers that can store any information necessary to have during the lighting calculations. Common buffers are the color, position, normal, and texture coordinate buffers.

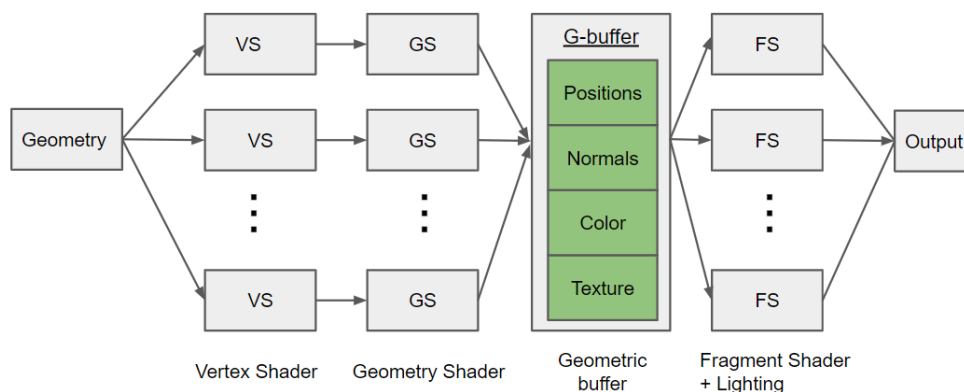


Figure 2.4: Deferred rendering pipeline.

Each buffer is usually a render target texture covering the whole screen where each position in the texture represents a pixel on the monitor. The big advantage of using deferred rendering is that the lighting calculations scale better with multiple light sources. If we have N light sources and M pixels in our monitor we would need to perform $N * M$ light calculations. The alternative used in naive forward rendering is to perform lighting calculations on every vertex in the scene, regardless of whether the vertices are in view or not. Some drawbacks with referred rendering includes difficulty handling translucent surfaces since the G-buffer only stores one surface per pixel and discards occluded surfaces which are important to keep when rendering translucent object such as clouds or glass [12]. The solution is usually to use a combination of the two methods, where we might do an initial pass with deferred rendering and then a forward pass to correctly render translucent objects.

2.1.4 Raytracing

As name suggests, raytracing is the act of following a ray and finding its intersection with objects in a space, see Figure 2.5. When light bounces of an object and reaches our eye, we are able to perceive the object. Raytracing reverses that concept by tracing rays from the camera's (*i.e.*, the eye's) point of view to find if and where it intersects with an object [17]. Finding intersections usually happens in one of two ways. Either you march along the ray, raymarching, until you hit an object or direct raytracing (see below). Raymarching could be expensive depending on how detailed you want your scene to be. Smaller steps allows the ray to intercept an object more precisely but leads to more steps per ray.

In some cases, where our objects are defined by a function, we can use something called direct raytracing [15]. This method involves solving the function:

$$\gamma(l) = s(x)$$

where $\gamma(l)$ is the ray function and $s(x)$ is the surface of the object we are looking at. Solving this can be faster then raymarching but has limited use if our objects cannot be described by a mathematical function.

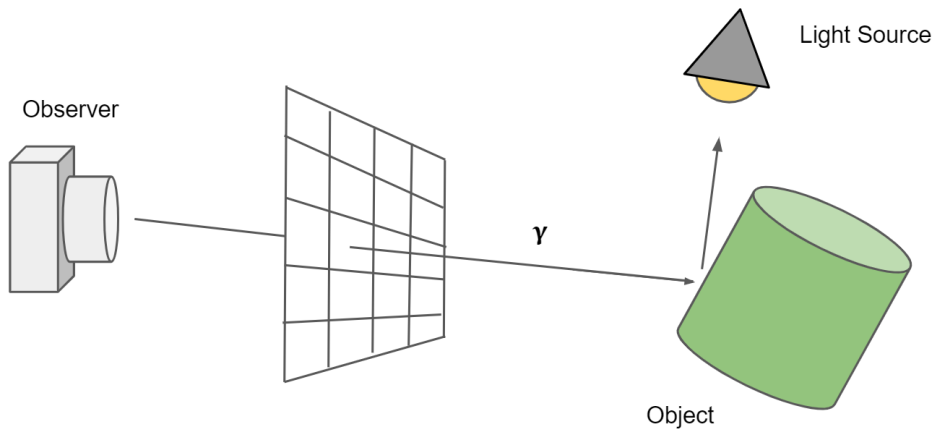


Figure 2.5: Raytracing example following a ray γ from observer to light source via an object in 3D space.

2.1.5 Line Rendering

Line rendering is a basic component in many visualizations, be it graph [2], particle trajectories [14], or streamline visualization [20]. The demands of different visualizations create different requirements for the line rendering but they often have to achieve a balance between visual fidelity and efficiency. Sections 2.1.5.1 to 2.1.5.3 give an overview of relevant line rendering methods and concepts.

2.1.5.1 Parametric curves and surfaces

Parametric curves and surfaces offer a detailed representation of a curve or surface without having to store and move all the geometric data that a mesh representation would demand [12]. A spline usually refers to a piecewise polynomial parametric function that has a continuous first derivative [6]. Splines in computer graphics can be used to compute curved lines or surfaces. A common spline used for this is the Bézier curve [12, 21]. A Bézier curve can be comprised of any number of control points but for simplicity we will examine the cubic Bézier curve with 4 points. A cubic Bézier curve is defined by:

$$C(t) = (1 - t)^3 * p_1 + (1 - t)^2 t * p_2 + 3(1 - t)t^2 * p_3 + t^3 * p_4$$

where $p_1 \dots p_4$ are points in space and $t \in [0, 1]$ and describes our position between p_1 and p_4 which are the start and end point in our curve respectively. Using this equation allows users to create their own curve by positioning the different points (see Figure 2.6).

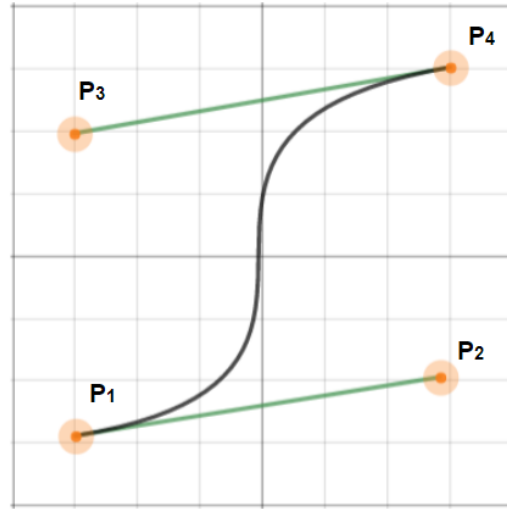


Figure 2.6: Cubic Bézier curve (black) with with points p_1, p_4 as starting and end point and p_2, p_3 as intermediate points. Curve drawn using Desmos [22].

Using segmented Bézier curves we can build something called a composite Bézier curve. A curve is known as a composite Bézier curve (γ) if the curve is a union of individual Bézier curves γ^l :

$$\gamma = \gamma^0 \cup \dots \cup \gamma^l [6]$$

So that the endpoint of the curve γ^i is the starting point of the curve γ^{i+1} for $i \in [0, l - 1]$.

The concept of Bézier curves can be extended to Bézier surfaces by interpolating in two dimensions instead of one. While a cubic Bézier curve is defined by four points and one variable 2.1.5.1, a biquadratic Bézier surface, or patch, is defined by 9 points and two variables [12].

2.1.5.2 Cylinder representation

One way of representing three-dimensional lines is to use a generalized cylinder. Sweeping a 2D contour along an axis can produce a variety of shapes. A generalized cylinder is defined by an arbitrary contour swept along an arbitrary 3D trajectory [23]. Tubes are a subset of generalized cylinders

using a sphere or circle as a contour [15], see the example shown in Figure 2.7. The trajectory can be represented in different ways, such as: splines [15] or a set of straight lines in a grid system [16].

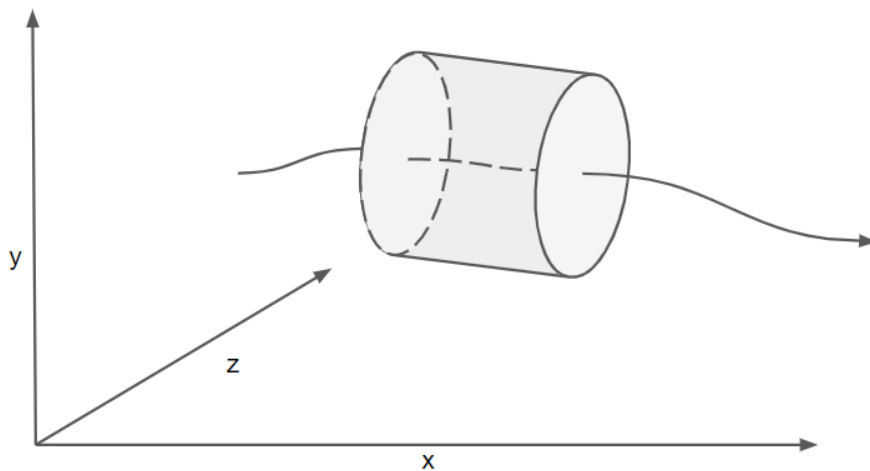


Figure 2.7: A segment of a generalized cylinder generated from moving a circle along a trajectory in 3D space.

2.1.5.3 Flat line representation

One alternative to generating and rendering actual tube geometry is to render something flat and then shading it to look like a tubular geometry. In 2004, M. Schirski, *et al.*, introduced the term ‘virtual tubelet’ [14] that does precisely that. The tubelet is based on quadrilateral which is a four sided polygon. Quadrilaterals are also be known as billboards. A texture is then applied to the quadrilateral in order to mimic the reflective properties of a cylinder.

A similar approach is to use something known as an imposter to replace the geometry. An imposter also uses a texture but the texture is calculated from the actual geometry and then applied to a billboard [24]. The texture is recalculated when an arbitrary threshold is met.

2.2 General-purpose computing on graphics processing units

The GPU has traditionally been used to render visuals as described in Section 2.1.1. Using them for any other purpose was bothersome and since the hardware was fixed in the graphics pipeline it required the user to use a graphics Application Programming Interface (API) [8]. Efforts to make the GPU more accessible to the scientific community led to Nvidia introducing CUDA in 2006 which promised a more general computing platform [25]. CUDA allows developers to fully utilize the GPU and compute anything, *i.e.*, outside the scope of the conventional graphics pipeline. This is what has become known as General-purpose computing on graphics processing units (GPGPU). Sections 2.2.1 and 2.2.2 will cover the basics of GPU architecture and the software that runs on them.

2.2.1 Hardware Architecture

While the CPU is designed to run sequential algorithms as efficiently as possible, the GPU is designed to execute explicitly parallel programs [8]. To achieve this, GPUs utilize a concept known as Single Instruction Multiple Data (SIMD). The main point is clearly stated in the name, *i.e.*, we perform the same instruction on different data at the same time. Using the graphics pipeline as an example we can think about how vertices in a scene need to be translated to accommodate the camera's position. In this scenario we perform the same operation on each vertex and we can increase performance by executing these calculations in parallel rather than sequentially (see Figure 2.8).

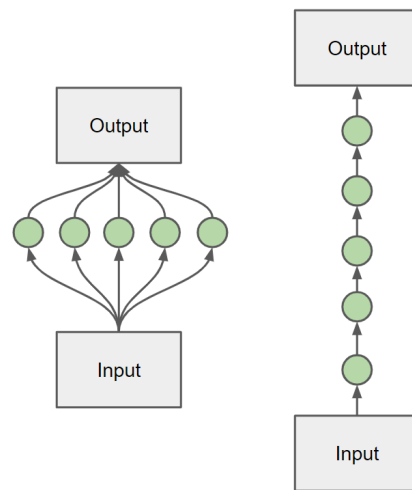


Figure 2.8: Simplification of Parallel and Sequential execution.

Figure 2.9 shows a basic GPU architecture. The GPU has processor units known as Streaming Multiprocessors (SMs) which are further divided into components. Every SM has a number of cores that are responsible for the operations in an algorithm. In the case of the Nvidia GTX 1050Ti GPU used in this project (see Table 3.1), it has 6 SMs and total of 768 cores. Comparing this to the Intel Core i7-8750H CPU described in the same table which has a total of 6 cores — we can begin to understand why the GPU usually excels in parallel computations. In order to fit as many cores as possible in a GPU the cores are generally smaller, simpler, and clocked at a lower frequency than the cores of the CPU.

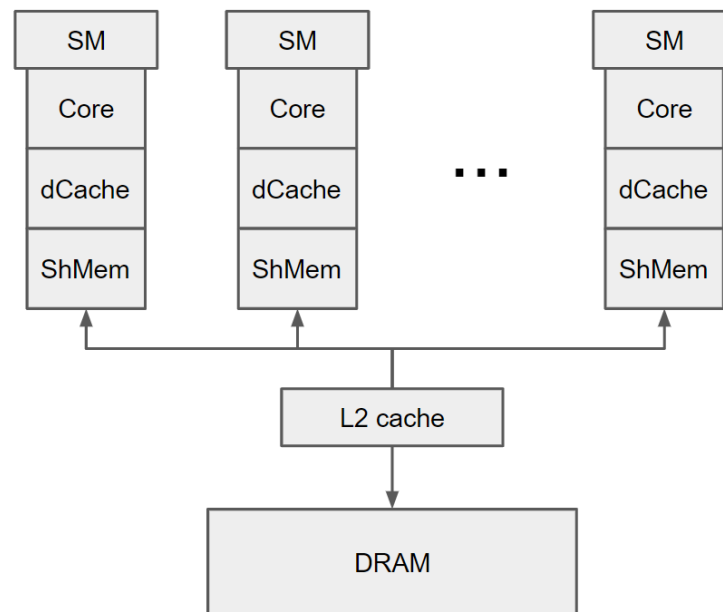


Figure 2.9: Basic GPU architecture, this figure was inspired by [8].

Like the CPU, the GPU has a hierarchy of memory units as seen in Figure 2.9. The Global memory labeled Dynamic Random Access Memory (DRAM) is the furthest from each core and is the biggest memory. The Nvidia GTX 1050Ti GPU has 4GB of DRAM. The closer you get to the cores, the smaller the memories. The L2 cache is 1024 kB while the direct cache (dCache), which is not shared among SMs, is 48 kB. Each SM can also have a shared memory (ShMem) which is a memory unit that is controllable by the developer, unlike the other caches. Although the global memory is much bigger than the caches closest to the cores, the caches, such as the dCache, can be up to x1000 times faster at retrieving instructions and data than the main memory [8].

2.2.2 Compute Shaders

Compute shaders, as the conventional shaders described in Section 2.1.1, are algorithms that runs on the GPU [26]. There is also difference between a conventional function implemented for the CPU and a function, or kernel as it is known in CUDA, implemented for the GPU. This section will use CUDA terminology to describe compute shader concepts.

CUDA introduced the concept of Single Instruction Multiple Threads (SIMT) which is a generalization of the earlier mentioned SIMD. A thread

executes a kernel and is the smallest granularity of computation in CUDA [8]. The idea behind SIMT is that the same instructions can be read by any number of threads while they operate on different data. What differs from SIMD is that threads can *diverge* and read different instructions. This can happen when, for instance, an `IF` statement occurs. This gives greater freedom to the developer but makes the program slower since it now runs sequentially. CUDA threads are grouped into *thread blocks* which are further grouped into *grids* (see Figure 2.10). Thread blocks execute independently which means that they do not coordinate amongst themselves [8]. A thread block will finish when all threads contained in it have completed its execution.

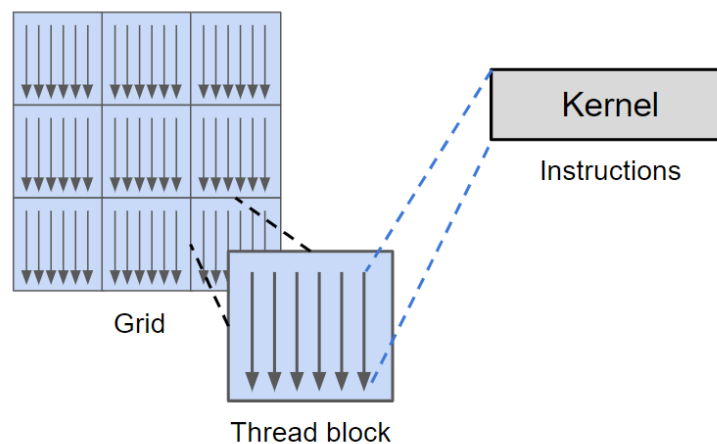


Figure 2.10: CUDA thread hierarchy where a thread is represented by an arrow. Figure inspired by [8].

In Algorithm 2.1 is a basic element-wise addition algorithm for the CPU and GPU. The `addCPU()` method loops through the array's elements, adds them together and populates the array `c`. Observing `addGPU()` on line 13 we can see that it declares `n` with `blockIndex`. This (`blockIndex`) is a global variable containing the index of the block the kernel is running in. When we call `addGPU()` on line 22 we pass in some configuration parameters. The first one specifies the number of blocks we want the kernel to run on and the second one is the number of threads in each block. The `addGPU()` kernel is then run on every block in parallel and we will get the same result as if we would have used `addCPU()` - but in much less time.

Listing 2.1: Pseudo code of GPU and CPU array addition

```

1 global int N = 10;
2 // CPU implementation of array addition
3 def addCPU(arr a, arr b, arr c){
4     int n = 0;
5     while{n < N}{
6         c[n] = a[n] + b[n];
7         n ++;
8     }
9     return;
10 }
11
12 // GPU implementation of array addition
13 def addGPU(arr a, arr b, arr c){
14     int n = blockIndex;
15     c[n] = a[n] + b[n];
16     return;
17 }
18
19 // calling of the CPU and GPU functions
20 array a, b, c = arrange();
21 addCPU(a, b, c);
22 addGPU<<N, 1>>(a, b, c);

```


2.3 Unreal Engine

Unreal Engine (UE) is a real-time 3D creation tool [9]. It was created as a game engine and is still thought of as one of the leading tools for creating immersive worlds where the visuals are the focus. Outside of the gaming industry, the engine is used for research as a visualization tool [2, 27], for visualizing traffic [28], and for crowd simulation [29]. Section 2.3.1 will cover the general rendering pipeline in Unreal Engine (UE) and Sections 2.3.2 and 2.3.3 explain some UE specifics regarding objects in scenes and UE's visual scripting language (respectively).

2.3.1 Rendering

Unreal Engine (UE) runs using a game thread and a render thread [9]. The render thread runs one or two frames behind the game thread and is responsible for drawing the scene in each frame. The game thread is responsible for CPU computations and user interaction. This section will focus mainly on the inner working of the render thread.

The process in the game thread of going from a scene to draw calls is shown in Figure 2.11. The rendering starts from *FPrimitiveSceneProxy* which is the render thread's representation of the scene to be rendered. The *FMeshBatch* decouples the user code from the mesh passes and contains everything needed to create the draw commands issued later. The *FMeshDrawCommand* is an interface between *FMeshBatch* and the Render Hardware Interface (RHI) and contain everything the RHI needs to know to issue draw commands to the underlying graphics API, such as what shaders to use, what resources should be included in those shaders, and the draw parameters. The RHI is an abstraction introduced by UE to be as system independent as possible. RHI is an abstraction layer above the platform specific graphics API which might be Vulkan, DirectX12, or Metal2. This allows the developer to write code that should work independently of the graphics platform that is running underneath the RHI.

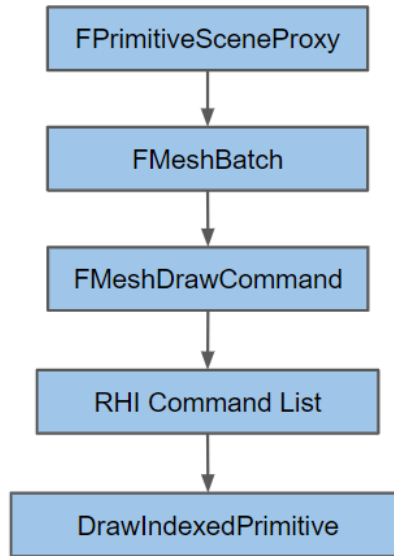


Figure 2.11: The path of a draw in UE. Inspiration from official documentation [9].

2.3.2 Actors

An Actor in UE is any object that can be placed into a level, such as a Camera, static mesh, or player start location. Actors support 3D transformations, such as translation, rotation, and scaling. In the case of this thesis, an actor is created to hold the information associated with the 3D tube models. This information includes the data representing the path of the tube as well as the mesh representing the 3D model of the tube. The actor also works as an interface as it handles user input, such as key presses. The functionality of an actor can be written in C++ as well as a visual scripting language called *Blueprint* which will be covered in Section 2.3.3. Every actor have a function called *Tick* which is called on every frame update.

2.3.3 Blueprint Visual Scripting

Blueprint is a node based visual scripting language used in UE. The system allows for easy code modification and can work as an interface between the UE editor and C++ code. Blueprint can be used as a way to trigger C++ code when there is user input and is used as such in this thesis project (See Figure 2.12). In the context of this thesis it is also used to toggle between the CPU and GPU

implementation of the algorithm.



Figure 2.12: Example of Blueprint.

2.4 Related work

This section surveys contemporary line rendering techniques together with methods used to accelerate visualization with a GPU.

2.4.1 Virtual Tubelets and Impostors

There are many cases where one wants to render a tubular geometry but, because of technical limitations, it has been infeasible to do so. These limitations could be limited computing power, or an issue of memory bandwidth. In those cases there has been a need to render something simpler and more efficient. As introduced in Section 2.1.5.3, virtual tubelets and impostors do this.

In 2004, M. Schirski, *et al.*, introduced the virtual tubelet and used it to visualize particle trajectories [14]. They compared their solution against rendering cylinders by measuring the frame rate when visualizing a fuel injection system on computer hardware ranging from newer to older hardware. They found that their method greatly outperforms tube rendering on the older systems but the difference diminishes with the newer systems. Redoing their

experiment today might reveal an even smaller difference in performance due to further hardware improvements. A problem with their method is the quality of the visual effect created. Since they use static textures to create the illusion of a tubular geometry it looks good from a certain angle. But we lose the sense of orientation when exploring the visualization in an immersive environment, since the lighting does not accommodate for the position of the observer.

More recently, in 2017, a similar but more sophisticated method was used by L. Nesi *et al.*, to visualize white brain matter fiber tracts from a diffusion tensor image scan [30]. They, as Schirski *et al.*, apply a texture to a billboard. However, their method is more sophisticated. Based on the camera's position they calculate a perspective P and affine transformation matrix M which they use to update the orientation of the billboards. They create a normal texture map which holds the normals of a cylinder. This generalized normal map is then applied to the line segment billboard *after* it has been multiplied by the upper left 3×3 affine transformation matrix. This gives us a normal map that corresponds to the normals that an actual cylinder would have. Using this information, they can use a standard lighting scheme to give credible lighting from any angle. Their solution reduces computational time by more than 50% and reduces memory usage by 900% as compared to rendering a tube geometry. They do not describe how the tube geometry was generated and rendered, so we can not know if the implementation they compare against was optimized or not.

2.4.2 Raytracing tubes

In a 2020 paper, B. Russig, *et al.*, explore the use of raytracing as an alternative rendering method for rendering [15]. Their method operates on a generalized cylinder where they move a sphere along a trajectory, *i.e.*, axis. The trajectory is represented with a Hermite spline. During rendering of the spline the tube is segmented into parts. Each segment is comprised of two nodes where each nodes holds information on position, radius, and their respective first derivatives. They encapsulate each tube segment into a bounding box which they then use in direct raytracing to find intersection points between the ray and the tube. Their solution was tested against a tessellation based method by rendering various datasets at different distances. They found that their solution scales better with the number of triangles in a scene but did not show any great advantages on smaller datasets, *i.e.*, with less than 10^5 triangles, but greatly outperforming the other method when the triangle count was above 10^7 .

Other raytracing methods include the one proposed by M. Kanzler, *et al.*,

that use a voxel based line representation instead of Hermite splines [16]. In this method the space is subdivided into a voxel based grid system. A ray is sent out for every pixel and then for every voxel we hit, we see if the voxel contains any lines or if we can skip this voxel. This method, like that proposed by B. Russig, *et al.*, in [15], performs better on larger datasets when compared to a standard rasterization method but struggles with smaller datasets and when rendering scenes with a lot of empty space.

2.4.3 Tessellation of parametric surfaces

With parametric representation of surfaces, we get a very precise description of a 3D object without having to store massive amounts of data. However, GPUs and graphics pipelines are often very specialized at handling geometric primitives, usually triangles. This means we have to convert these parametric surfaces into meshes via tessellation [31]. Early implementations of real-time surface tessellation include the work by A. Rockwood, *et al.*, in 1989 [32]. Here they subdivide a surface into Bézier patches and then tessellate these patch into triangles of a user defined size. The subdivision and tessellation of surfaces were performed on the CPU while the GPU handles the resulting triangle mesh. However, having to transfer data between the CPU and GPU is a costly procedure and the method could be sped up by performing the tessellation on the GPU instead.

R. Concheiro, *et al.*, [31, 33] proposed two different approaches to perform tessellation directly on the GPU. These two different methods use vertex shader tessellation (VST) and geometry shader tessellation (GST) respectively. This limits the size of the geometry having to be moved from the CPU to the GPU, since most of the the mesh is created within the graphics pipeline. Some drawbacks of this method are that neither the vertex shader nor the geometry shader are tailored to produce a massive amount of geometry.

Tessellation is becoming more commonplace in computer graphics and during the 2010s major GPU manufacturers, such as AMD and Nvidia, started to incorporate tessellation stages into their rendering pipeline and gave developers easier access to it [34, 35].

By utilizing the extended capabilities of modern graphical pipelines G. Nunes, *et al.*, [36] overcame the limitations that R. Concheiro, *et al.*, [31] faced. They were able to efficiently render generalized cylinders by generating geometry around a discrete curve. The generation of geometric primitives is performed using hardware tessellation which takes advantage of the new hull, tessellator, and domain stages. By performing the tessellation in a dedicated

pipeline they could generate 12.3 million triangles at an above 100 fps on a Nvidia GTX 480.

2.4.4 GPU Acceleration in visualization

Many modern visualization schemes utilize the computing power of a GPU. For example, P. Mi, *et al.*, created a GPU centered design for large multidimensional datasets [37]. They highlight two important aspects when designing a GPU powered visualization system. The first involves storing and handling the raw data directly on the GPU. This minimizes the communication between CPU and GPU, which is often costly, and allows the system to take full advantage of the GPU's high memory bandwidth and parallel computing power. Interactions are processed by the CPU and instructions are sent to the GPU. A simplified flow chart of the process is shown in Figure 2.13.

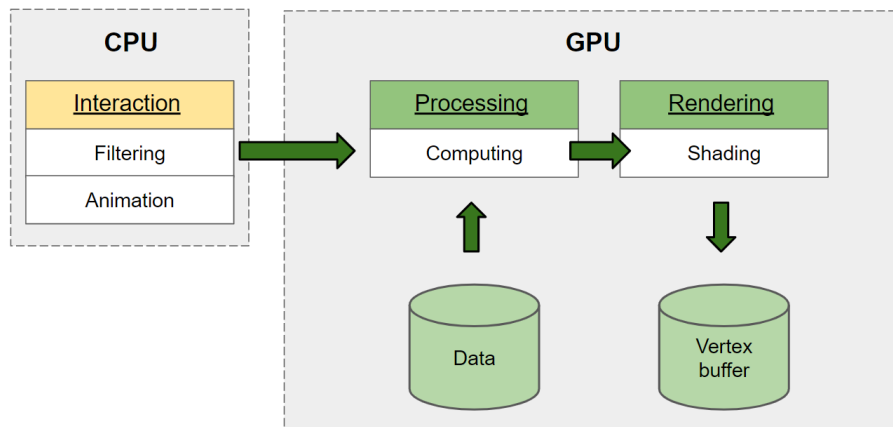


Figure 2.13: Flow chart of a GPU-centric design. User interaction is handled by the CPU while the data processing and rendering is all handled by the GPU.
 . Figure inspired by [37]

Restricting factors of this approach are sometimes the limited memory of the GPU and the rigid manner in which it accesses this memory. Compressing the data and organizing it in appropriate ways works for some data but cannot scale to very large datasets without adding more GPU memory.

A. Panagiotidis, *et al.*, created a modular tool for graph layout and rendering on the GPU [38]. Their method involves loading and processing the graph data on the CPU before sending the data to the GPU. The GPU handles the layout, rendering, and further processing of the data without ever having to send the data back to the CPU again. This follows the principle proposed

by P. Mi, *et al.*, in [37] and minimizes the amount of data needing to be passed between CPU and GPU memories.

2.4.5 Graph visualization

Research on massive graph visualization usually focuses on increasing sense making and promoting easy and efficient interaction. P. Mi, *et al.*, introduced a force-directed layout method which allows for systems with million of nodes [39]. Their solution utilizes three GPU kernels that compute the attractive and repulsive forces for each node and then updates the node's position.

O. Kwon, *et al.*, [2] explored how to move from conventional 2D visualization on a monitor into the realm of immersive environments in VR. Their paper introduces rendering techniques for greater clarity and improves interactivity with novel highlighting techniques that utilizes depth cues. Findings from their user study shows that their graph visualization specialized for immersive environments allows the user to complete tasks faster than with conventional 2D visualization. The accuracy of the study participants was better with the biggest graph in the study which had 297 nodes and 2359 edges but there was no significant difference with the smaller graphs used in their study.

2.4.6 Summary of Graph visualization

None of these solutions use actual tubular geometry to generate lines. In contrast, my method will try to create an efficient solution to render generalized cylinders such that the method can be used in similar visualizations.

2.5 Summary

Chapter 2 has introduced the reader to the necessary background knowledge in computer graphics needed to understand the rest of this thesis. Furthermore, it presented the reader with relevant methods for line rendering that have been used in different scenarios (see Table 2.1). M. Schirski, *et al.*, [40] and L. Nesi, *et al.*, [30] both presented very efficient methods for line rendering. However, their methods produce 2D results which are less relevant in the immersive visualization research conducted at Visualiseringscenter C [11]. M. Kanzler, *et al.*, [16] and B. Russig, *et al.*, [15] used some form of parametric curve to define the tubes they rendered. Their methods are relevant

to this thesis since I will use Bézier curves to represent my curves. However, using raycasting is not as relevant to me since it was shown to be slower with smaller data sets and when rendering dense opaque line sets. G. Nunes *et al.*, [36] proposed a tessellation based method for tube rendering from a discrete curve. Their method takes advantage of the tessellator stage discussed in Section 2.1.2 to create their geometry while my method will utilize compute shaders. One benefit of doing this could be that we skip the step of having to create patch primitives to send to the tessellator and thus we can circumvent the tessellation stage entirely.

In summary, my method aims to produce a more flexible solution than M. Kanzler, *et al.*, and B. Russig, *et al.*, and a result more fitting for immersive visualization than M. Schirski, *et al.*, and L. Nesi, *et al.* My method will utilize compute shaders to compute the geometry from splines entirely on the GPU.

Table 2.1: Comparison of related line rendering techniques and my own

Paper	Method	Efficient	Visuals
M. Schirski, <i>et al.</i> , [40]	Billboards	All	2D
L. Nesi, <i>et al.</i> , [30]	Impostors	All	2D
B. Russig, <i>et al.</i> , [15]	Raycasting	Large	3D
M. Kanzler, <i>et al.</i> , [16]	Raycasting	Large	3D
G. Nunes, <i>et al.</i> , [36]	Tessellation	All	3D
Woxler	Compute Shaders	All	3D

Chapter 3

Method

This chapter describes the general research process used in this thesis (Section 3.1). Section 3.2 explains how the experiments were performed and what results were produced. The section also presents the parameters used in the experiments and other relevant variables, such as hardware and software used. Both the method used for the experiments as well as the data produced are analysed in terms of validity and reliability in Section 3.3. The actual implementation of the algorithms used in the experiments will be covered in Chapter 4.

3.1 Research Process

The research process for this thesis project have been done to achieve the goals stated in Section 1.4. Figure 3.1 outlines the steps conducted to carry out this research. Reviewing earlier work and methods to find suitable ways to achieve the desired outcome also revealed a way to evaluate the proposed tool. After surveying the scientific literature and in conjunction with Visualiseringscenter C, it was decided that this project would use compute shaders in UE as a method of geometry generation. UE was chosen as the framework for this thesis project because of its prior use in visualization research, as well as its advanced options and tools for providing an impressive visual result. In order to develop a tool for UE, an investigation into its rendering pipeline was required. This investigation generated the foundational knowledge needed to create an efficient visualization system and the general pipeline structure was described in Section 2.3.1.

The next step was to develop an appropriate compute shader and make it work in the UE framework. This procedure took a considerable amount of

time since the documentation was sparse and the support for compute shaders in UE is limited. The rendering pipeline survey was used to complete this task and the outcome from the development phase can be seen in Chapter 4.

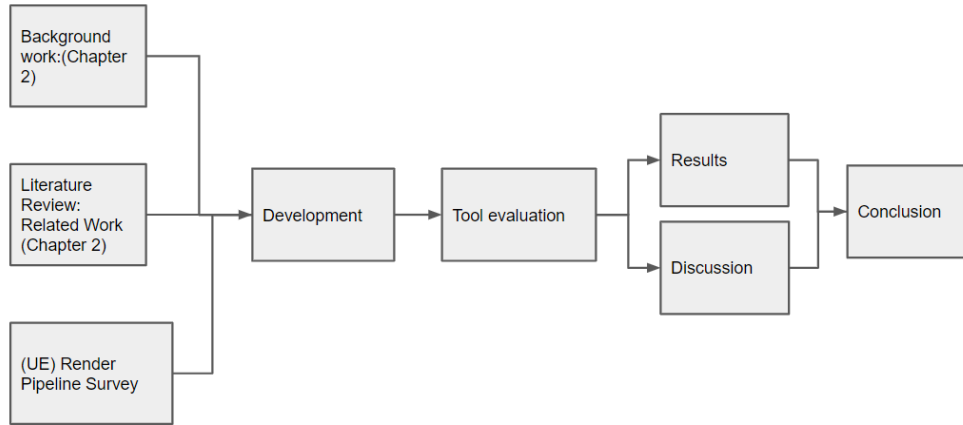


Figure 3.1: Flow Diagram of the research process for this thesis.

The algorithm implemented using compute shaders will be compared to a sequential implementation that runs on the CPU. This algorithm was created in order to have baseline results to compare the compute shader implementation to. Given a longer time, frame variations of the compute shader algorithm would have been created and compared against as well.

Each experiment uses the time it took to generate and render the tube segments as a metric since it was the most relevant and a common way of evaluating efficient rendering techniques. The choice of this metric is supported by the related work where we can see that timing is a common metric for evaluating experiments related to computer graphics (see Section 2.4). Each experiment will be purely quantitative and no effort was made to analyse the visual results. The parameters used will be further described in Section 3.2.2 and the input data for each experiment is covered in Section 3.2.3. Analysis of the results from each experiment is presented in Chapter 5.

3.2 Experiment Design

This section covers the experiments performed to answer the research question stated in Section 1.2.1. This section also gives the parameters necessary to repeat the experiments. Section 3.2.1 gives an overview of how each experiment was conducted. Sections 3.2.2 and 3.2.3 cover the parameters

and data acquisition (respectively). For an in depth description of the implementation of the two algorithms used, see Section 4.1.

3.2.1 Experimental Setup

To find out if one algorithm outperforms another, I designed an experiment such that both algorithms are given the same data and then use that data to create models that contain the same features. The input data used is described in Section 3.2.3. The output of the algorithms are used to create 3D models and both the algorithms create vertices, triangle indices, normals, and tangent vectors for the models.

As explained in Section 3.2.2 some of the parameters of the experiment are altered to investigate if either of the algorithms outperform the other in some specific configurations. For every configuration, 100 frames are recorded and execution data is saved into a csv-file (see Section 3.2.4). Additionally, once every frame, the position of the model is updated and new model data is generated using the active algorithm. Using the data collected from the experiments a quantitative analysis is performed to find if there is any difference in performance of the two algorithms.

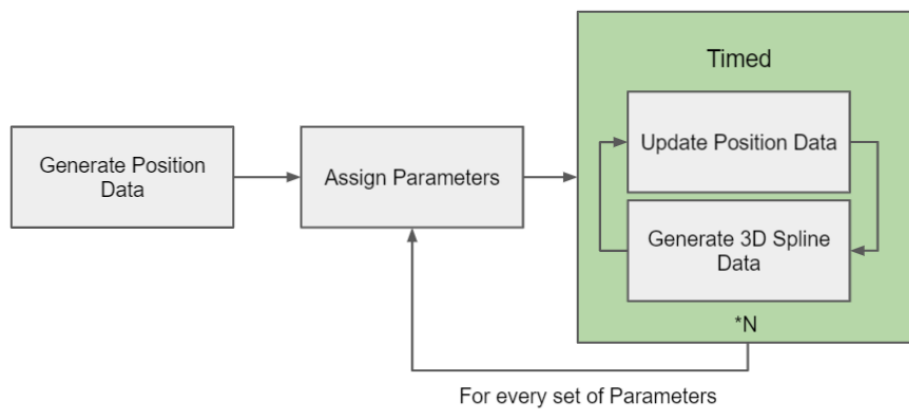


Figure 3.2: Overview of experimental design. In the figure, “*N” represents N repetitions of the experiment.

3.2.2 Experimental Parameters

The aim of the experiments was to examine whether, and under what conditions my compute shader implementation of an algorithm on the GPU

would outperform an algorithm creating the same data on the CPU. To do this I explore a set of configurations that ranged from low to high level of detail (LOD) as well as a varying number of line segments. The LOD is defined by the number of vertices that surround each centroid that constitutes the line segments (*e.g.*, as shown in Figure 4.2 on page 39). Increasing or lowering the number of vertices will lead to a more or less detailed end product. The second variable is the number of segments that is to be rendered in a scene. The total number of triangles generated is dependent on these two variables in such a way that

$$N = LOD * LineSegments * 2$$

where N is the total number of triangles. Both of the parameters are varied to determine if one or the other has a bigger effect or if the total number of triangles is the major deciding factor in the rendering time. The exact parameters used can be seen in Section 5.1. The range of LOD was chosen from the lowest possible value and up to where a satisfactory visual quality was achieved (See Figure 3.3). The number of lines were varied up until and above similar experiments, such as in [39] where there were 10^6 nodes in a graph. The maximum number of triangles generated and rendered in this case will be $3.6 * 10^7$ when we input 10^6 line segments to be rendered with a $LOD = 18$.

When running the experiments the *smooth frame rate* and *Vsync* options have both been disabled. These options help make projects run smoother on different machines but they also limit the performance of the application to match the update rate of your particular monitor which is undesirable when doing experiments. I have also disabled screen space ambient occlusion (SSAO), bloom and atmospheric fog which, during initial testing, tied up a great deal of the GPU. Since all these effects are purely visual effects, I felt no need to have it activated during testing.

3.2.3 Experimental Data

To perform geometry generation requires data that describes the path the tube segments will follow. This data is a set of vertices which we will call *centroids*. The centroid vertices act as a path on which the tube segments are generated. Each centroid is at the center of a start and end point of a tube segment, hence the name.

The centroids for these experiments will not be derived from real-world data but instead synthesized. Since we are primarily interested in the time required to generate and render these tubes, the orientation and location of these tubes does not impact the performance of the algorithm as we have to

generate the same number of triangles regardless of orientation and location. However, if the algorithm were to be optimized to create more or fewer triangles given some condition, such as occlusion or distance, then it would be relevant to test the algorithm on real-world data.

The centroids that will act as the path on which the tube segments will be generated around are created in a spiral configuration, specifically N tube segments are generated containing $N + 1$ centroids where $N > 0$. Each centroid C_i is given a position in 3D-space, and each line segment L_i is created such that L_i is a connection between C_i and C_{i+1} where $i \in [0, N]$. The positions of the centroids are determined by a starting position $P_0 = \{x_0, y_0, z_0\}$ and then the centroids are assigned a position P_i where

$$P_i = x_0 + scale_x * t * \cos(t), y_0 + scale_y * t * \sin(t), z_0 + scale_z * t$$

The variable t starts at zero and is incremented for each centroid with a user defined value t_{Step} which in the case of the experiments was set to $t_{Step} = 0,06283$ which gives a pleasing visual result. The scale is also user defined and set to $scale = \{25, 25, 25\}$. These user defined variables only affects the visual outputs of the experiment and are chosen in a semi-arbitrary fashion. An example of a spiral generated with these parameters and a radius $r = 20$ is shown in Figure 3.3.

As mentioned in Section 3.2.1 the data is updated once every frame. This is done by rotating each centroid around the Z-axis of the model, which is the axis going straight upwards in the scene. The x and y values of each vertex are updated as:

$$\begin{bmatrix} x' & y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

while the z-value is unchanged. The results is a rotating spiral updated each frame.

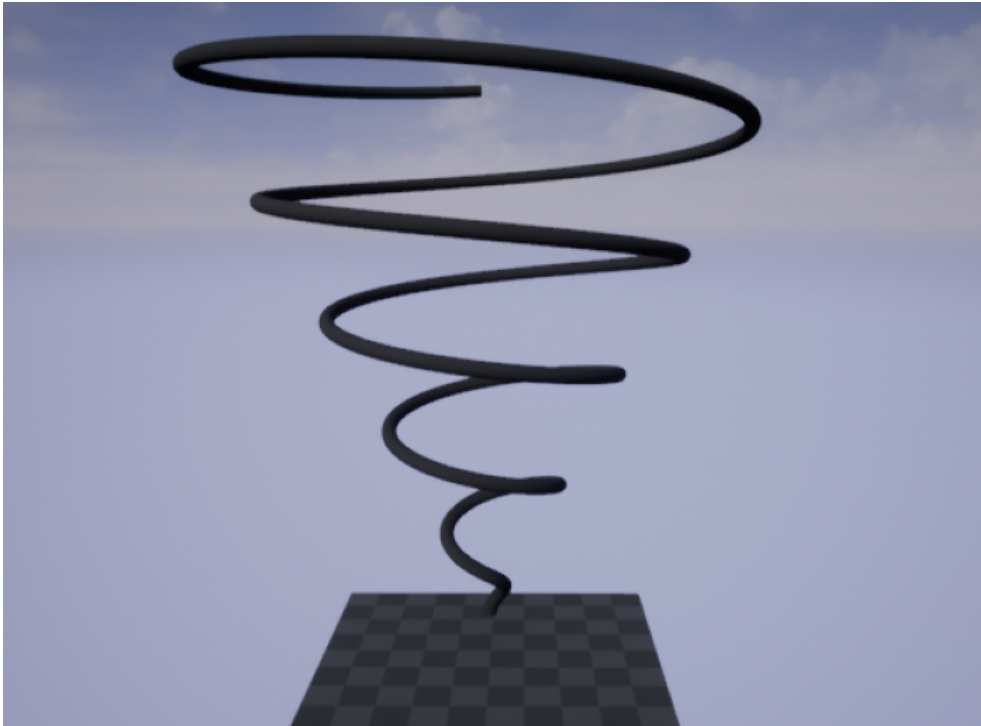


Figure 3.3: Spiral generated using 500 centroids (*i.e.*, $N = 499$) and parameters $t_{step} = 0,06283$, $scale = \{25, 25, 25\}$, and $r = 20$.

3.2.4 Collected Data

A common metric when doing research in computer graphics is fps. For example, this is used in [2, 40, 16] amongst others. This thesis will instead focus on the render time of each frame and the decomposition of each frame window. Data on fps will be collected but will only be used as a secondary metric and will not be used to answer the research question. As noted previously, the data collected is purely quantitative as it is the time it takes to generate and render a set of triangles to the screen.

The timing is recorded using UE's internal profiling tool [41]. This allows for recording of frames and all the data associated with each individual frame. The profiler allows the user to see the execution time of function calls, shaders, render and game thread *etc.* This data allows us to gain more insight, not simply on *how* the algorithms perform but also *why* it performs this way. The time measurements are recorded in ms with three decimal places.

The most important measure of performance will be the time it takes to render the frames on which the model is rendered. This will be measured for both the CPU and GPU implementation. Both implementations will include the execution time of the game and render thread. The compute shader implementation will include the execution time of the shader generating the geometry as well as the total time spent on the GPU.

3.2.5 Hardware and Software

The Dell XPS 7570 hardware used in this project is described in Table 3.1. The software used is Unreal Engine v4.26 [9] and Visual Studio Community 2019 [42]. For the CPU implementation of the algorithm, I utilized the plugin run time mesh component (RMC) v4.1 [43] which will be further discussed in Section 4.2.1.

Table 3.1: Hardware specifications of the Dell XPS 7570 [44, 45, 46]

Hardware	CPU	GPU
Model	Intel Core i7-8750H	Nvidia GTX 1050Ti
Clock speed	2.2 GHz	1.29 GHz
RAM-size	16 GB	4 GB
RAM-bandwidth	2.67 GB/s	112.13 GB/s

3.3 Method and Data Validity and Reliability

To ensure that the experiment is measuring the same thing when executing the two different algorithms, both algorithms are given the same input data. Additionally, the vertices produced by the two algorithms are compared to ensure that the output from them is equivalent.

As stated previously, the only data collected is the amount of time it takes to generate and render a given set of input data to the screen. Using the native profiler tool gives a reliable way of recording the time it takes to render each frame as well as an overview of where the computation time is going. This allows us to verify if any change in render time is correlated to any specific process.

Since the experiment aims to find differences in execution time between the two contending algorithms, accurately timing the execution of them is essential. One thing that the experiment does not take into account is the impact of different graphical APIs. Since the test was run on a single hardware platform (as described above) with a single API (DirectX 12) the testing cannot show how the algorithms would perform on other systems. Doing the experiments on different machines with different hardware and graphical APIs would have given a more general answer instead of the specific answer in the case of the current machine. However, testing the performance on other platforms was outside the scope of this project (see Section 1.6).

To ensure reliability of the experiment each scenario used for an experiment is run for 100 frames and the average result is presented. The two algorithms will also be tested after one another in an effort to mitigate the effect of the software environment, *i.e.*, no difference in active applications or programs on the machine that the experiment runs on.

For each configuration of parameters tested, a Mann-Whitney U test will be used to see if any difference in the results are statistically significant. The results from these tests will be presented in Section 5.2 alongside the major results from the experiments.

Chapter 4

Development

In Section 4.1 the compute shader implementation of the tube rendering software is described in depth. Section 4.2 briefly explains the CPU implementation of the tube rendering software.

4.1 Compute Shader Implementation

This section describes how the compute shader was implemented in UE and explains the structure of the implementation. A basic overview of the structure is explained in Section 4.1.1. Section 4.1.2 describes each step in greater depth and explains how the shader generates the tubular geometry. The complete code used for geometry generation in the compute shader can be found in Appendix A.1.

4.1.1 Shader Structure Overview

The compute shader code itself is written in High Level Scripting Language (HLSL) and generates the data necessary to create 3D models. This data is then output to buffers which are handled by the graphics pipeline. In order for UE to use the shader, the compute shader needs to be declared so that the engine understands it. Declaring the shader both instantiates it and enables the engine to find and use it. The game thread can then invoke the shader and provide it with the required parameters. The shader's output is inserted into the render thread and can then be drawn together with any other geometry. For a visual representation of the structure, see Figure 4.1

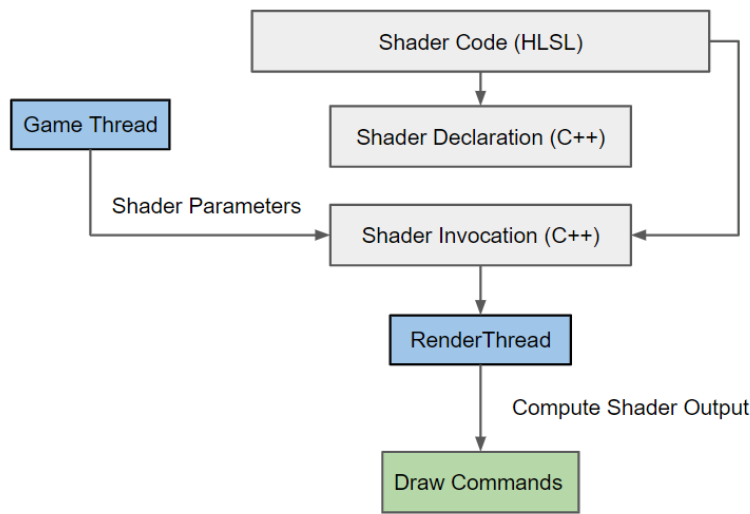


Figure 4.1: Compute shader overview in UE.

4.1.2 Implementation

Section 4.1.2.1 describes how the shader generates geometry. An improvement to the basic implementation is presented in Section 4.1.2.1 where some visual artefacts are handled. Sections 4.1.2.3 and 4.1.2.4 explains how to enable the shader for use in the UE framework and how to invoke it (respectively).

4.1.2.1 Compute Shader first implementation

The shader file is written in HLSL, a shader language created by Microsoft in order to write shaders for DirectX [47]. The file itself has a Unreal Engine shader file (usf) extension which is the file extension used by shaders in UE.

The input for the shader consists of position data representing line segments. This is an array of vectors containing the positions of the centroids of the tube segments that are to be created. These centroids acts as a path for which we generate the tube geometry. The shader input also includes additional information that allows user control of the visual aspects of the tube segment, such as the tube radius and a LOD factor. LOD can be altered by increasing or decreasing the number of vertices used to represent the tube segments. For example, if we use three vertices the shader will generate three

vertices around each centroid, and these will be used to generate the necessary triangles (see Figure 4.2). The shader also needs configuration data indicating how it should execute the code with respect to the size of the thread batch and number of threads per call.

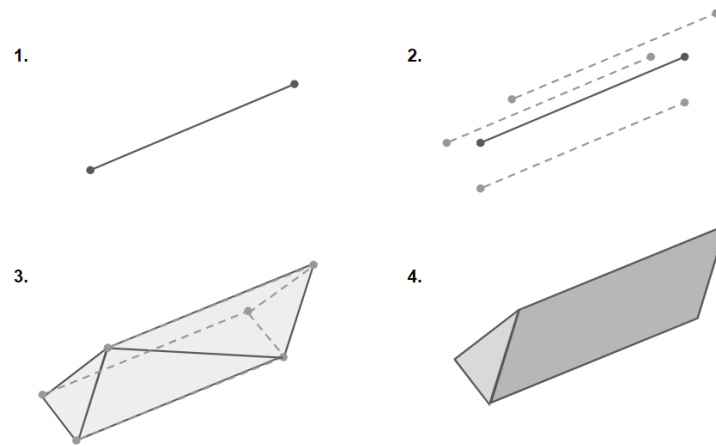


Figure 4.2: Generating a tube segment with three vertices representing the tube segment. Step one is a line segment between two vertices in 3D space, where each vertex acts a centroid. The next step shows how the shader has added three new vertices around each of the original vertices. In step three we have created polygons from the newly generated vertices and the last step is the rendered shape.

The output of the shader is two different buffers. The first buffer is a vertex buffer containing a UE structure called `DynamicMeshVertex`. This structure contains five variables: position, color, texture coordinates, and two tangent vectors X and Y. The second output buffer is the index buffer which tells the renderer how to construct the triangles from the vertex buffer.

Each thread in the compute shader is responsible for creating a single vertex around a single centroid and to populate the index buffer with information of how to construct a single quad containing two triangles for each created vertex.

Every invocation of the shader has information about the current thread configuration. As explained in Section 2.2.2, threads are grouped into blocks which are further divided into grids. The shader knows the index of the thread block, the index of the thread in this block, and the global thread index of the shader dispatch. The invocation also has access to the size of the thread block grids. Using this information we can calculate the index of the current vertex

of the vertex input buffer. This index is used to access both the current vertex V_{center} and the next vertex in the line V_{next} according to the equations:

$$V_{Center} = vertex_input_buffer[idx];$$

$$V_{Next} = vertex_input_buffer[idx + 1];$$

The normalized vector between the current and next point is used to define the direction tangent of the current tube segment and is stored in a variable T_X :

$$T_X = Normalize(V_{Next} - V_{Center});$$

The tangent is then used as a basis vector to create two vectors B and C which are both of unit length and perpendicular to each other and T_X . These vectors identify the plane that is used to determine the position of a new vertex that sits on the circumference of a circle surrounding the center vertex. The position of the new vertex V_{New} are calculated as:

$$V_{New} = V_{Center} + B * r * \cos(\theta) + r * \sin(\theta) * C$$

where θ is determined such that each vertex generated on the circle is of equal distance from each other and the radius r is defined by the user (see Figure 4.3). This approach of creating new vertices do generate some visual artefacts which are discussed and solved in Section 4.1.2.2.

The surface normal associated with the new vertex is the normalized vector from the center vertex and the new vertex:

$$V_{Normal} = Normalize(V_{New} - V_{Center})$$

After generating a new vertex, the shader then adds the correct indices that make up the triangles associated with said vertex. This process uses the same information as used when finding the index used to access the correct centroid.

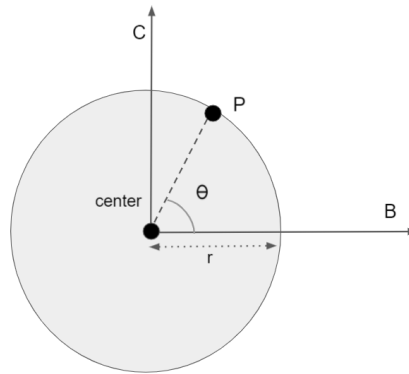


Figure 4.3: A vertex P is positioned on the circle of a radius r around a center vertex.

4.1.2.2 Improved compute shader

A problem with the basic shader implementation is that each new vertex generated at the end of a segment lies on the plane for which the direction tangent vector is a normal. Since all segments are created in the same manner, this creates an unwanted visual artefact at the point where tube segments meet (see Figure 4.4). One way of fixing this is to "cut" the segments at the meeting point at an appropriate angle.

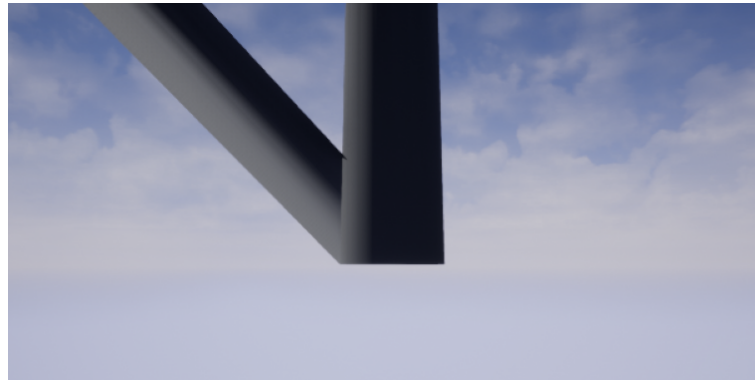


Figure 4.4: Visual artefact of tube segments meeting.

The direction vector of the segment T_X is calculated in the same way as described in the previous section. As an extra step we also calculate the

directional vector of the next segment in line T_{X+1} as:

$$T_{X+1} = \text{Normalize}(\text{vertex_input_buffer}[\text{idx} + 2] - V_{\text{next}})$$

where idx and V_{next} are declared in the same way as in the previous section. A new vector $T_{\text{BiSegment}}$ is then calculated as the normalized vector in the middle of T_X and T_{X+1} calculated as:

$$T_{\text{BiSegment}} = \text{Normalize}(T_{X+1} - T_X)$$

When letting a plane intersect a cylinder the intersection forms an ellipse. In the previous naïve approach that ellipse was a special case scenario in which the ellipse was a circle. To know the positions of points on the edge of an ellipse, we need to know its minor and major axis. The minor axis of the ellipse is going to be the radius of the tube segment we are cutting. The major axis is calculated as:

$$\text{major_axis} = r / \sin(\theta_{\text{BiSegment}});$$

$\theta_{\text{BiSegment}}$ is the angle at which the cylinder is cut in relation to its directional tangent vector. This is the same as the angle between T_X and $T_{\text{BiSegment}}$ and is calculated as:

$$\theta_{\text{BiSegment}} = \text{acos}(\text{Dot}(-T_{\text{BiSegment}}, T_X));$$

where $\text{Dot}(V_1, V_2)$ returns the dot product of the input vectors V_1 and V_2 . The B and C vectors that stretch the plane on which the new vertices are placed on are defined as:

$$B = T_{\text{BiSegment}}$$

$$C = \text{Cross}(T_{\text{BiSegment}}, T_X)$$

where $\text{Cross}(V_1, V_2)$ returns the cross product of vectors V_1 and V_2 . Any new vertex that lies on edge the ellipse can now be calculated as:

$$V_{\text{new}} = V_{\text{Center}} + B * r * \cos(\theta) + \text{major_axis} * \sin(\theta) * C$$

This new approach gets rid of the visual artefact which presented itself in the more naïve solution (see Figure 4.5). However, it require more computations to calculate the extra variables needed. These extra computations impact the compute shader's implementation more than the sequential implementation since the sequential implementation can save T_{X+1} and set T_X to its value

when needed (see Section 4.2). Since each thread must be independent of others the compute shader implementation must calculate both T_X and T_{X+1} for each thread dispatched.

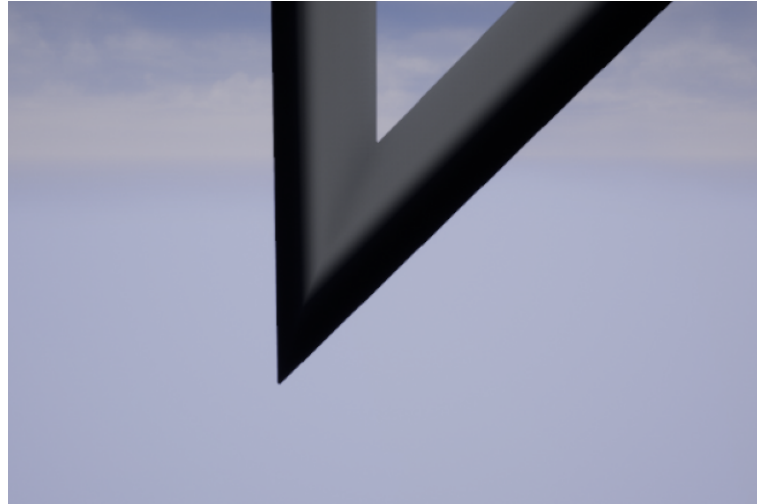


Figure 4.5: Meeting tube segments without visual artefact.

4.1.2.3 Declaration

As previously stated we need to declare the shader in order for UE to recognize and use it. This is done by creating a C++ class associated with the usf file and this class acts as an interface between the engine and the HLSL. The class we created inherits from the `UE FGlobalShader` class, thus allowing us to use it as an UE shader. In the associated C++ file we use the macro:

```
DECLARE_SHADER_TYPE(shaderClass, Global)
```

which creates a shader of class *shaderClass* that is recognizable by the engine. This also tells us that the shader is global, thus it ends up in the global shader map and is applied on a fixed geometry and does not need a material to be used [9]. For the engine to find and use the shader code, we also need to use another macro:

```
IMPLEMENT_SHADER_TYPE(shaderClass, path, function, type)
```

which tells the engine where the shader is located, what function in the shader we intend to invoke, and what type of shader it is (vertex, pixel, or compute shader).

Aside from declaring the shader itself, we also need to declare the parameters to be used by the shader when invoked. Shader classes in UE can declare their parameters using:

```
PARAMETER_STRUCT()
```

which allows UE to recognize the shader's parameters.

4.1.2.4 Invocation

Upon executing the program we initialize a vertex factory which is responsible for passing vertex data from the compute shader to a vertex shader.

Invocation of the shader is performed from the game thread on the CPU by sending user input to an UE actor which in turn generates segment data according to the user's defined parameters. This user data contains the LOD, radius, and number of tube segments to be generated. We must also provide instructions on how the centroids should be positioned, which in our case is the spiral configuration explained in Section 3.2.3. The data is then sent to the render thread and the compute shader is dispatched from there. The tube segment data is accessed by the shader, which uses it to generate all the information necessary to render the appropriate 3D model.

4.2 CPU Implementation

This section describes the tube generation algorithm that is designed to run sequentially on the CPU. It also describes the Run time mesh component (RMC) plugin used for mesh generation as well as giving an overview of how the sequential algorithm differs from the compute shader algorithm.

4.2.1 Run Time Mesh Component

RMC is a plugin for UE designed for generating and rendering meshes [43]. As the name implies it allows efficient updating of the mesh at run time which is something we want for information visualization, *e.g.*, if we would want time dependent movement or movement triggered by user as seen in [2, 39]. The algorithm used to generate the tubes is similar to the compute shader algorithm with the difference that it runs sequentially. It loops through each vertex in the vertex input buffer and generates the tube geometry associated with that vertex.

RMC has several distinct parts, each responsible for a certain part of the process. `URunTimeMeshComponent` is the main component and it

allows you to place a mesh in a scene and interact with it as you would any mesh in UE. `URuntimeMeshProvider` provides the mesh object with renderable mesh data, configurations, and collision setting and data. The version of this provider created for this thesis inherits from the base class `URuntimeMeshProvider` and is responsible for creating tube model data and passing it to `URunTimeMesh`. `URunTimeMesh` is another part of RMC and acts as a data carrier and copies mesh data to the GPU memory. The provider made for this project does not create any collision data (at this time) but that might be desirable in the future.

4.2.2 Algorithm

The CPU implementation has a few advantages over the GPU implementation. Examining the compute shader implementation we see that it needs to compute the basis vectors B and C once per thread. Remember that these basis vectors stretch the plane on which we generate the new vertices for each centroid. In the CPU implementation this only happens **once** per centroid. The same reasoning holds true for the directional tangent vector T_X which we calculate **once** per thread in the compute shader rather than **once** per centroid in the CPU implementation. For a pseudo code implementation of the sequential algorithm see Algorithm 1. Note that the pseudo code described is only a basic implementation of the algorithm. For the code in full, see Appendix A.2.

Algorithm 1 Pseudo code sequential implementation**Input :**

Vertex_In // Array of vertices
 LOD // Number of edges around tube
 r // Radius of tubes

Output :

Vertex_Out // vertex data for tube mesh
 Triangles_Out // Triangle index buffer

```

1 Main(){  $T_{X+1} = \text{Normalize}(\text{vertex\_buffer\_in}[1] - \text{vertex\_buffer\_in}[0])$ 
   $a_1 = \text{radius}$  for  $idx = 0$  to  $\text{Vertex\_In.Size}() - 1$  do
    // Get the current centroid
2     $V_{Center} = \text{vertex\_in}[idx]$  // The direction tangent vector is
    the previous  $T_{X+1}$  tangent
3     $T_X = T_{X+1}$  // The direction tangent vector for the next
    segment is calculated
4     $T_{X+1} = \text{Normalize}(\text{vertex\_in}[idx + 2] - \text{vertex\_in}[idx + 1])$  // The
    bi segment vector is the vector that lies in between
    the current and next direction tangent vector
5     $T_{Bi\_Segment} = \text{Normalize}(T_{X+1} - T_X)$  // The half angle between
     $T_X$  and  $T_{X+1}$ 
6     $\theta_{Bi\_Segment} = \arccos(\text{Dot}(-T_X, T_{Bi\_Segment}))$  //  $a_2$  is the major
    axis of the ellipsis created at the meeting point of
    the current and next tube segment
7     $a_2 = \text{radius} / \sin(\theta_{Bi\_Segment})$  // Reset theta
8     $\theta = 0$  for  $j = 0$  to  $LOD$  do
      // New vertex that lies on the outer shell of the
      tube end is created and added to the out buffer
9       $V_{New} = V_{Center} + B * a_1 * \cos(\theta) + a_2 * \sin(\theta) * C$ 
       $\text{vertex\_Out.Add}(V_{New})$  // The two triangles  $Tri_1, Tri_2$ 
      associated with each new vertex is added to the
      output triangle buffer
10      $\text{Triangles\_Out.Add}(Tri_1)$   $\text{Triangles\_Out.Add}(Tri_2)$ 
      // Increment the theta angle
11      $\theta += 2 * \pi / LOD$ 
12 }
```

Chapter 5

Results and Analysis

This chapter presents the major results from the experiments as well as an analysis of these results and the validity of them. Section 5.1 presents the results and Section 5.2 analyze their statistical validity. Section 5.3 presents an analysis of the results and the conclusions based on them.

5.1 Major results

Testing the algorithms at different LOD reveals whether LOD was a factor that significantly impacts the difference in performance of the two algorithms. Table 5.1 show the average frame rendering times using a LOD of 3 and 9. Both LODs were tested and recorded during 100 frames with

$$N \in [10^1, 10^2, 10^3, 10^4, 10^5, 2 * 10^5, 4 * 10^5, 6 * 10^5, 8 * 10^5, 10^6]$$

where N is the number of tube segments generated. The experiments were also conducted using $LOD = 18$, but because of the unreliability of the results, they are omitted from this thesis (see Section 6.3). Table 5.1 presents the number of segments generated, together with the average frame render time for both the GPU and CPU implementation using the two LODs. Note that all of the timings are given in units of ms. Table 5.2 shows how the two algorithms react to altering the LOD by comparing different LOD using the same number of segments. Table 5.4 and 5.3 presents an overview of two individual algorithms (respectively). Figure 5.3 show how the LOD affects the generation and rendering time when using the sequential and compute shader implementation of the algorithm. Figure 5.2 and 5.1 are visualizations of Table 5.4 and 5.3 (respectively).

Table 5.1: Average results in ms from the performed experiment with $LOD = 3$ (a) and 9 (b)

(a) $LOD = 3$

Segments	GPU	CPU
10^1	5.02	2.50
10^2	2.92	3.97
10^3	4.97	15.97
10^4	8.80	6.38
10^5	3.91	35.62
$2 * 10^5$	6.34	69.58
$4 * 10^5$	12.59	130.53
$6 * 10^5$	16.12	189.81
$8 * 10^5$	20.92	267.17
10^6	25.75	395.48

(b) $LOD = 9$

Segments	GPU	CPU
10^1	3.20	2.50
10^2	7.47	5.63
10^3	3.03	8.17
10^4	10.17	11.46
10^5	7.25	85.29
$2 * 10^5$	7.38	166.48
$4 * 10^5$	11.38	373.26
$6 * 10^5$	17.05	602.03
$8 * 10^5$	21.42	853.81
10^6	25.70	1099.85

Table 5.2: Average results comparing the time it took to render a frame when generating N segments with $LOD = \{3, 9\}$ in ms on (a) the CPU and (b) the GPU

(a) CPU		
Segments	$LOD = 3$	$LOD = 9$
10^1	2.50	2.50
10^2	3.97	5.63
10^3	15.97	8.17
10^4	6.38	11.46
10^5	35.62	85.29
$2 * 10^5$	69.58	166.48
$4 * 10^5$	130.53	373.26
$6 * 10^5$	189.81	602.03
$8 * 10^5$	267.17	853.81
10^6	395.48	1099.85

(b) GPU		
Segments	$LOD = 3$	$LOD = 9$
10^1	5.02	3.20
10^2	2.92	7.74
10^3	4.97	3.03
10^4	8.80	10.17
10^5	3.91	7.25
$2 * 10^5$	6.34	7.38
$4 * 10^5$	12.59	11.38
$6 * 10^5$	16.12	17.05
$8 * 10^5$	20.92	21.42
10^6	25.75	25.70

Table 5.3: Average results obtained from recording the compute shader algorithm during 100 frames. The table show the average execution time of various processes in ms, when generating N segments.

(a) LOD = 3

Segments	Frame	RenderThread	Game Thread	GPU Total	Spline
10	5.02	2.86	1.20	3.14	0.02
100	2.92	2.60	1.17	2.60	0.01
1000	4.97	2.86	1.26	2.96	0.01
10 000	8.80	5.49	1.59	8.77	0.30
100 000	3.91	3.21	3.86	3.50	0.37
200 000	6.34	4.24	6.34	5.32	1.13
400 000	12.59	5.41	11.66	11.13	2.23
600 000	16.12	7.59	16.11	12.64	3.80
800 000	20.92	7.87	20.91	16.78	4.84
1 000 000	25.75	12.73	25.74	20.02	6.31

(b) LOD = 9

Segments	Frame	RenderThread	Game Thread	GPU Total	Spline
10	3.20	2.75	1.16	2.87	0.01
100	7.47	4.74	1.28	6.01	0.01
1000	3.03	2.65	1.18	2.71	0.02
10 000	10.17	5.90	1.52	8.96	0.33
100 000	7.25	4.64	3.73	6.70	0.92
200 000	7.38	3.99	6.21	5.59	1.53
400 000	11.38	5.75	11.31	10.18	3.65
600 000	17.05	7.39	16.27	15.23	6.32
800 000	21.42	9.46	21.35	18.68	7.28
1 000 000	25.70	12.00	25.69	23.31	9.31

Table 5.4: Presenting the summary of results from running the sequential implementation. The table shows the average execution time of various processes in ms, when generating N segments

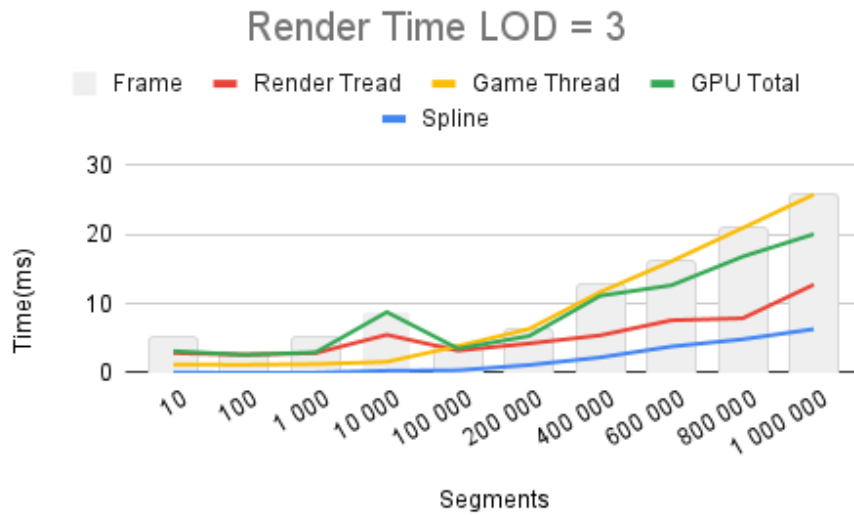
(a) LOD = 3

Segments	Frame Time	Render Thread	Game Thread
10	2.50	2.27	1.15
100	3.97	3.21	1.31
1000	15.97	8.80	2.10
10 000	6.38	2.12	6.37
100 000	35.62	3.15	35.62
200 000	69.58	3.93	69.58
400 000	130.53	3.18	130.54
600 000	189.81	22.60	189.80
800 000	267.17	29.62	267.16
1 000 000	395.48	31.14	395.47

(b) LOD = 9

Segments	Frame Time	Render Thread	Game Thread
10	2.50	2.28	1.23
100	5.63	3.86	1.44
1000	8.17	4.74	2.30
10 000	11.46	3.41	10.59
100 000	85.29	9.43	85.28
200 000	166.48	24.87	166.47
400 000	373.26	37.94	373.25
600 000	602.03	52.72	602.02
800 000	853.81	60.47	853.80
1 000 000	1099.85	80.55	1099.84

(a) LOD = 3



(b) LOD = 9

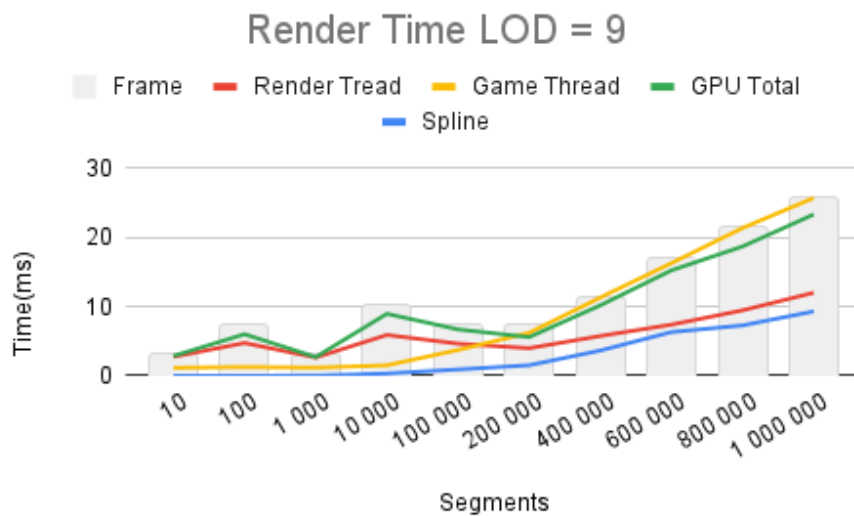
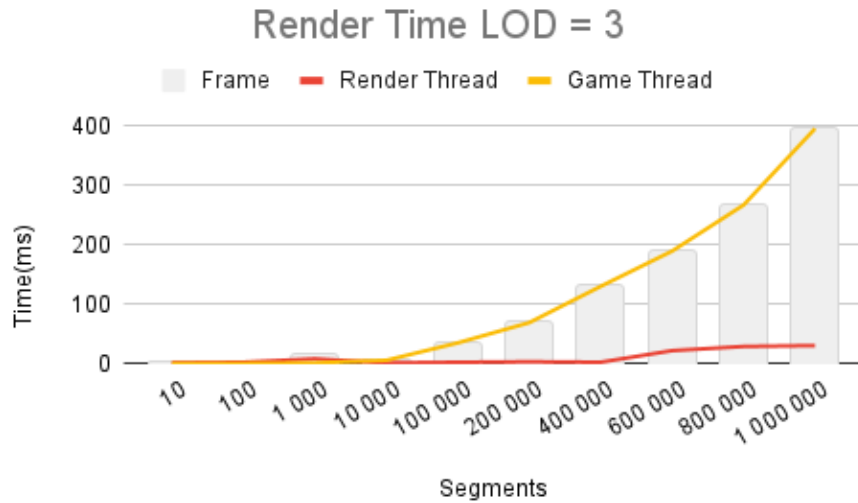


Figure 5.1: Presenting the average frame rendering time when using the compute shader implementation and LOD = 3 (a) and 9 (b). The bar shows the average rendering time while the different curves showcase the average execution time of different processes.

(a) LOD = 3



(b) LOD = 9

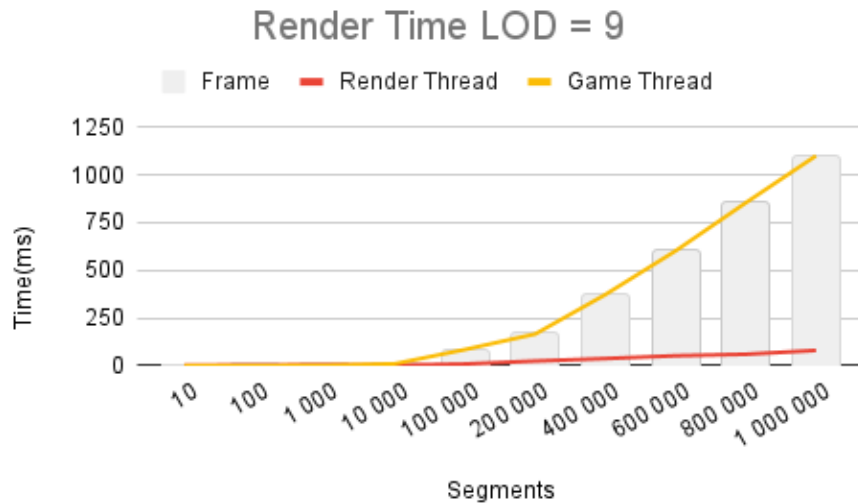
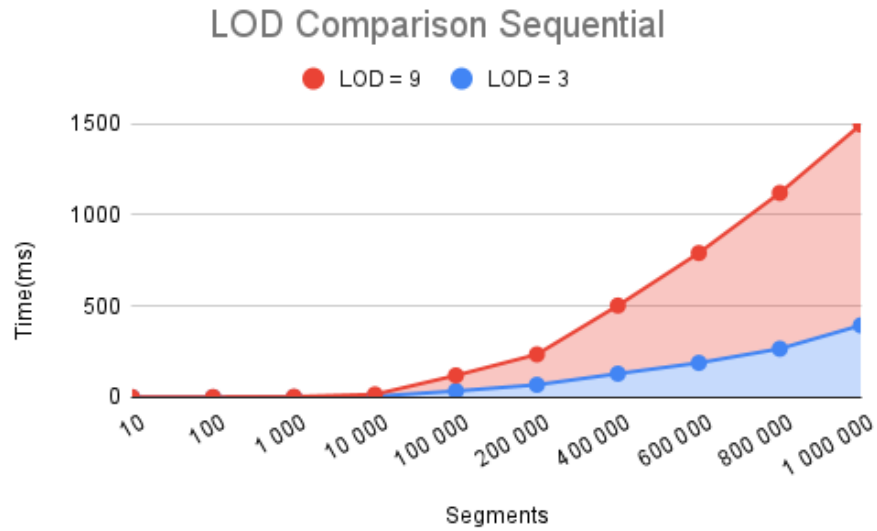
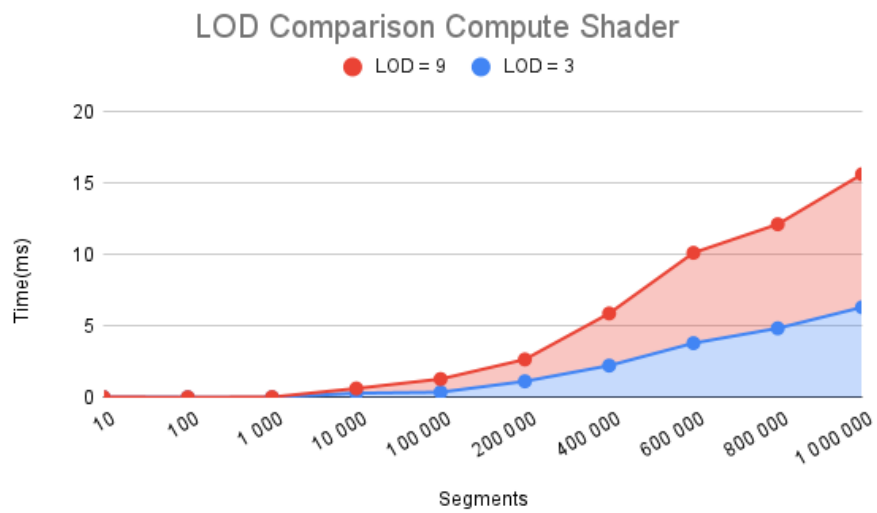


Figure 5.2: Presenting the average frame rendering time when using the sequential implementation and LOD = 3 (a) and 9 (b). The bar shows the average rendering time while the different curves showcase the average execution time of different processes.



(a) Sequential



(b) Compute shader

Figure 5.3: Presenting execution time of sequential (a) and compute shader (b) implementation when using different LODs.

5.2 Statistical validation

In order to know if any differences found in the results are statistically significant, a Mann-Whitney U test using the original results from running the experiments was performed. Note that, when the p-value = 0.00, it does not mean that the value is zero. It is simply because the value is rounded. Table 5.5 shows the P-values obtained when performing the test on the data collected from executing the sequential and compute shader algorithms with the same configurations, *i.e.*, the same LOD and number of segments. A p-value < 0.05 means that we reject our null hypothesis H_0 with a confidence level of 95%. H_0 is defined as:

$$\mu_1 = \mu_2$$

where μ is the mean value of a data set.

Table 5.5: Presenting the P-values obtained from Mann-Whitney U tests performed on the results from executing the CPU and GPU implementations of the algorithm. The X-axis shows the LOD used and the Y-axis shows the number of tube segments generated in the experiment.

Segments	LOD = 3	LOD = 9
10^1	0.01	0.45
10^2	0.01	0.00
10^3	0.00	0.00
10^4	0.00	0.00
10^5	0.00	0.00
$2 * 10^5$	0.00	0.00
$4 * 10^5$	0.00	0.00
$6 * 10^5$	0.00	0.00
$8 * 10^5$	0.00	0.00
10^6	0.00	0.00

Table 5.6 shows the results from a Mann-Whitney U test comparing algorithms with themselves, generating the same number of segments while altering the LOD. This is done in order to determine if altering LOD significantly impacts the generation and rendering time. The results from these tests will be discussed in the following section.

Table 5.6: Presenting the p-value results from a Mann-Whitney U test, performed on the results obtained when running the same algorithms with varying LOD using the sequential and compute shader implementation

Segments	Sequential	Compute Shader
10^1	0.46	0.24
10^2	0.00	0.00
10^3	0.02	0.15
10^4	0.00	0.00
10^5	0.00	0.05
$2 * 10^5$	0.00	0.01
$4 * 10^5$	0.00	0.00
$6 * 10^5$	0.00	0.00
$8 * 10^5$	0.00	0.00
10^6	0.00	0.00

5.3 Analysis

This section presents an analysis of the results showed in Section 5.1. The section is divided into a comparison of the two algorithms that were tested in Section 5.3.1. Section 5.3.2 and 5.3.3 analyses the two algorithms individually and examines why they perform the way they do.

5.3.1 Comparison of sequential and compute shader implementation

Looking at Table 5.2 we can see that the CPU variant of the algorithm outperforms the compute shader implementation at a few configurations. When $LOD = 3$ the sequential algorithm is faster when generating 10^1 and 10^4 segments. At $LOD = 9$ it is faster when generating 10^1 and 10^2 segments. Looking at Table 5.6 we can see that these differences are statistically verified, using the Mann-Whitney U Test, for all but one configuration, $LOD = 9$ when generating 10 segments. This gives some indication that the sequential algorithm is, in some cases, faster than the compute shader implementation. However, it is also true that when generating models as small as this, *i.e.*, $segments < 10^4$, hiccups and stalls in the system impact the the results a fair deal. Looking at Table 5.1 we can see that both algorithms perform rather unpredictable when generating smaller data sets. This makes it to draw

any conclusions regarding the superiority of any one method when working with smaller sets, *i.e.*, $segments < 10^4$. It is also reasonable to speculate that, in practical use scenarios, one would want to enable the **smooth frame rate** and **v.sync** options in UE. In that case the user would not notice any difference between the implementations since both of them, when generating $\leq 10^4$ segments, run at at least 60 fps, and the system would limit the fps to the update rate of the monitor. In order to conclude if any meaningful difference in performance is present when generating smaller models, more tests are proposed. This was unfortunately outside the scope of this thesis project.

When generating $\geq 10^5$ segments the compute shader implementation starts to greatly outperform the sequential one. This is mainly due to the game thread in the sequential algorithm. When the execution time of the game thread is close to the frame render time, the system is most likely bound by the game thread [48]. Looking at Table 5.4 and Figure 5.2, we can see this occurring with the sequential algorithm. Since the geometry is created in the game thread, it makes sense for the game thread to be the dominant factor of the sequential implementation.

Observing the render thread execution time in Table 5.4 we can see that, when generating $> 1.8 * 10^6$ triangles, it has a spike in execution time. The reason for this is unclear and with more time it would have been investigated further (see Section 6.3). However, if we compare the execution time of the rendering thread, in the two implementations, we can observe that, even if the generation of the geometry was completely free of computational cost, the cost of the render thread would make the sequential algorithm slower than the compute shader implementation when generating $> 1.8 * 10^6$ triangles. In the case of the compute shader implementation, only positional data of the centroid vertices needs to be passed to the GPU. Though, in the case of the sequential algorithm we need to pass complete mesh data which includes positions, normals, tangent vectors, and colors. This means that the sequential algorithm will reach the bottleneck of the render thread faster than the compute shader implementation.

Examining the results presented in Table 5.1 we can directly compare the two implementations. As stated earlier, the compute shader implementation starts to greatly outperform the sequential algorithm when generating $\geq 10^5$ segments. Looking at Table 5.7 we can see that the compute shader algorithm is $15\times$ and $42\times$ times faster than the sequential algorithm when generating 10^6 segments using $LOD = \{3, 9\}$ (respectively). Moreover, the compute shader implementation seem to handle an increase in LOD better than the sequential

implementation. Examining Table 5.2 and Figure 5.3 confirms this suspicion since increasing the LOD in the sequential implementation increases rendering time by a factor $> 2 \times$ when generating $> 10^4$ segments. This behaviour can not be seen when examining the frame rendering time of the compute shader implementation. Moreover, when examining the execution time of the execution time of the compute shader itself, increasing the LOD increase it by a factor $< 2 \times$. Except for a single case when generating 10^5 segments.

Table 5.7: Presenting the results from dividing the frame rendering time of the sequential implementation with the compute shader implementation. This is done for $\text{LOD} = \{3, 9\}$

Segments	LOD = 3	LOD = 9
10^1	0.50	0.78
10^2	1.36	0.75
10^3	3.21	2.70
10^4	0.73	1.13
10^5	9.11	11.76
$2 * 10^5$	10.97	22.56
$4 * 10^5$	10.37	32.80
$6 * 10^5$	11.77	35.31
$8 * 10^5$	12.77	39.86
10^6	15.36	42.80

5.3.2 Sequential implementation

Examining Table 5.4 and Figure 5.3a we can see that when generating $> 10^4$ segments, the game thread is the limiting factor for the sequential implementation. Since the game thread is the limiting factor of this implementation we would need to decrease the execution time of the mesh generation algorithm in order to increase its performance. A possible solution which I discuss in Section 6.3 is to utilize multi-threading, in order to take advantage of modern CPU's parallel computing power. It should be made clear that the game thread includes other processes such as the rotating of the centroids each frame. However, if we look at the execution time of the game thread of the compute shader in Table 5.3, which includes the rotational algorithm, we can see that it has a relatively small impact on the total execution time of the game thread of the sequential algorithm ($\sim 1\%$ for 10^6 segments).

5.3.3 Compute shader implementation

Examining Figure 5.1 we can see that, like with the sequential implementation, the compute shader implementation is bound by the game thread when generating $2 * 10^5$ segments. The game thread is responsible for rotating the centroids each frame, which is most likely the reason why it acts as a bottleneck. When analyzing the compute shader implementation I will mostly focus on the execution time of the total GPU time and the spline compute shader, which can be seen in Table 5.3 and Figure 5.1.

Assuming that we could decrease the execution time of the game thread, the GPU would be the bounding factor of the compute shader implementation. The spline generation shader is part of the GPU process and when generating bigger models it is also the dominating process of the GPU, with the two following being the pre- and base-pass. These render passes include the execution of conventional shaders and depth calculations. Since most visual effects have been turned off for the experiments, it is not likely that we could minimize the effect of these any further, especially for use in any real world scenario. This is because we would most likely want to employ some kind of material or special shader onto the model which would increase the execution time of these passes. Ways of improving the compute shader itself is discussed in Section 6.3.

For examining the cost of the algorithm on the GPU memory bandwidth, we can make some calculations. The output of the shader is an array of `FDynamicMeshVertices`, which are 88 B each (see Table 5.8a). We also output an array of integers storing the polygon information needed to render the mesh. Each vertex has two associated polygons, which mean that we, for each vertex, store 24 B of additional data (see Table 5.8b). If we generate 10^6 segments, each which creates 9 vertices, we generate:

$$Vertex_data = 10^6 * 9 * 88 \text{ B} = 0.792 \text{ GB}$$

$$Polygon_data = 10^6 * 9 * 24 \text{ B} = 0.216 \text{ GB}$$

In total we then generate 1.008 GB of data each frame. Looking at Table 5.3 we can see that the average frame time when generating data of that size is 23.31 ms, thus we generate (per second):

$$1.008 * \frac{1000}{23.31} \simeq 43.24 \text{ GB}$$

The bandwidth of the GPU on which the experiments are run GPU is

112.13 GB/s. This means that we, at most, utilize $\sim 39\%$ of the memory bandwidth if we send the data once. Following the path of data in the GPU is a complicated task and not something that was in the scope of this thesis. However it is reasonable to think that doubling the data size, for instance if we set the LOD to 18, that we would start running into issues with memory bandwidth. In order to dig a little deeper into this issue, I used the tool *Nvidia Nsight Graphics* [49] which allows the user to monitor GPU processes. With this tool, the performance of the GPU was monitored while generating 10^6 segments with LOD = 9. The most demanding processes were shown to be the primitive distributors (PD), which are in charge of index-buffer loads, and the Video Random Access Memory (VRAM). The metric used for these are speed of light (SOL), which indicates how close to the theoretical maximum rate each process operates at. The VRAM had a SOL percentage of 56.2% while the PD was at 85.9%. The SM, in charge of shaders, were only at 14.8% indicating that we are underutilized the computational power of the GPU. The numbers also indicate that, as the system is currently built, we are likely to run out of memory bandwidth before we run out of computing power if we were to increase the LOD or size of the data sets.

Table 5.8: FDynamicMeshVertex and Triangle data structure components and their size

(a) FDynamicMeshVertex

Variable	Type	Size(B)
Position	float3	12
Textures	float2[8]	64
Color	uint	4
Tangent X	uint	4
Tangent Y	uint	4

(b) Triangle

Variable	Type	Size(B)
Triangle Indices	uint	4*3

Chapter 6

Conclusions and Future work

This chapter will conclude this thesis and will cover the conclusions made from the results in Section 6.1. Section 6.3 presents how the project could have been improved, as well as how one could build upon it. In Section 6.2 the major limitations of this thesis is presented and the chapter ends with the final reflections in Section 6.4.

6.1 Conclusions

Looking at the results presented and analysed in previous sections we can start answering the research question stated in Section 1.2.1 as:

What is the performance impact on time efficiency of adapting a cylinder mesh generation algorithm, from running on the CPU to executing on the GPU of a mid-level graphics workstation? (In 2021, we define a mid-level graphics workstation as an Intel Core i7-8750H as the CPU with an Nvidia GeForce GTX 1050Ti 4G as the GPU).

The analysis of the results show a potential improvement in generation and rendering time of a factor $> 40\times$ for the compute shader implementation running on the GPU. We can also observe that the impact on time efficiency increases with bigger data sets. For small data sets $N < 10^4$, where N is the total number of segments generated, a small to non-existent difference can be found and verified. Another characteristic of the compute shader implementation is that the parallel algorithm run on the GPU is better at handling an increase in LOD. Where an increase in LOD on the sequential implementation resulted in an $> 2\times$ increase in frame render time, the same

action resulted in an increase $< 2\times$ using the compute shader implementation. This thesis was carried out with two subgoals in mind stated in Section 1.4 as

- Subgoal 1** Create a tool capable of visualizing 3D tubes for use in information visualization at Visualiseringscenter C.
- Subgoal 2** Implement and evaluate different optimization techniques for GPU and CPU line generation.

Subgoal number one has been achieved, although with some restrictions. The resulting product from this project is a plugin for UE that can generate 3D tube geometry from a set of centroids. As of now it can only run in Windows and some extra functionality has yet to be implemented, as is discussed in Section 6.3.

6.2 Limitations

The most limiting factor of this thesis project was that of my own ability as well as the complexity of the problem. Building compute shaders in Unreal Engine (UE) turned out to be more complex and time consuming than anticipated. This thesis project was started without any prior knowledge of UE and a significant amount of time was spent learning, rather than producing working code. This limited my efforts and gave me less time to optimize both the compute shader implementation **and** the sequential implementation. This led to a less optimized end product which has room for improvement. Overall, there are some points that could have been improved upon, and those points will be discussed in Section 6.3.

As mentioned briefly in Section 5.1 the experiments performed included testing the two implementations with $LOD = 19$ as well as the presented cases of 3 and 9. However, the results from the tests using a $LOD = 18$ were unpredictable and when examining the data it seemed that when generating too many triangles, the compute shader only executed every other frame. The reason for this (as of this time) is unknown, and there was not enough time to investigate the problem further. It was instead decided that these results should be omitted, thus focus was instead on the outcome of the more reliable experiments. The negative consequence, is that it is harder to draw conclusions on how well the algorithms handles an even more detailed model.

6.3 Future work

Due to the breadth of the problem and the limitations stated in Section 6.2, only some of the initial goals have been met. The following sections will focus on the remaining issues that should be addressed in future work. Section 6.3.1 describes the features and techniques which were intended to be implemented at the start of the project, but due to lack of expertise and time were not able to be integrated. Section 6.3.2 enumerates several possible ways one could build upon this project.

6.3.1 What has been left undone?

This thesis project stated some initial goals in Section 1.4. This section will explain some of the ways that these goals have not been achieved.

6.3.1.1 Optimization

The thesis did not manage to fulfill the second subgoal set at the start of the project namely:

Implement and evaluate different optimization techniques for GPU and CPU line generation.

With enough time, various ways of optimization of both the algorithm and data structures would have been investigated and implemented. Possible properties one could experiment with include:

- Input and output data buffers for compute shader,
- Thread configuration, and
- Pre-computing certain elements on the CPU.

The data buffers used to pass data in to and out from the compute shaders might not be optimal. Shaders in UE can use a multitude of different buffers depending on their purpose. Trying out different configurations might result in an increase in efficiency.

The batch sized used when dispatching the compute shader might work well for one size of data but sub-optimally for another. With more time, one could experiment with different thread configurations and perhaps make a system in which the thread configuration was dependent on the data size, in order to best take advantage of the underlying hardware and software.

As mentioned in Section 4.1.2 the design and natural limitations of the parallel algorithm means that the solution, in total, needs to compute more

data since every thread must be independent. One way of improving the overall solution might be to pre-compute some essential data on the CPU and then send this data as input to the GPU. For instance, one could calculate the directional tangent vector of each individual tube segment which in the current approach is recalculated for each thread.

For the sequential implementation, one could investigate the potential of multi-threading as a method for improving performance. One could utilize multi threading by dividing the input position data into several batches and let each thread handle their own batch. Another possible solution is employ multi-threading at the point where new vertices are generated, thus a single thread is responsible for generating a single new vertex and associated triangle indices.

There exists many different approaches when it comes to optimization and the proposed methods are simply those methods that were considered during this Master's thesis project. Others that wish to continue this project would certainly find additional approaches to optimization suited to their knowledge and purpose.

6.3.1.2 Tool features

The end product of this thesis was meant to be a tool able to generate tubular geometry in an efficient manner using compute shaders in UE. The basic functionality of the tool (at present) is that it can:

- Generate splines efficiently given path data,
- Produce normal vectors associated with each vector, and
- Produce satisfactory visual results at corners.

However, the tool has points where it can be improved. In Section 1.6 it was stated that the visual qualities of the generated tube models were not one of the main goals of the project. Even though it was not a requirement of the project, it would have been advantageous to properly apply UV-coordinates to the model. This in itself would not improve the aesthetics of the model but would have facilitated use of textured materials. This feature could have been used to, for instance, create a color gradient from start to finish of each start and endpoint of a line segment or to give the model a textured surface such as metal, wood, *etc.*

Another useful feature which could be implemented, is integration with UE's current spline editor system. The current spline system allows the user to easily create and adapt splines for different uses. A good feature of the tool created for this thesis would be if it could use UE's spline creation tool and

then choose to convert that spline data into a format readable by the spline tool created for this project. This approach would take advantage of the user interaction system native to UE and the compute shader generation created in this project.

As mentioned in Section 4.2, as of now, the code does not compute any collision boundaries. In order to facilitate user interaction with the model this would be needed. Implementing collision boundaries for this kind of model is not a great effort but was outside the scope of this thesis project.

Overall the tool works well enough for the purpose of this thesis but as a tool for efficient information visualisation some minor improvements would have to be implemented.

6.3.2 Next obvious things to be done

One way of building upon this project would be to evaluate it using real world data in a real world application. This could include performing user studies where users, in a Virtual environment (VE) try to gain insight into graph data of various sizes. This would require implementing some of the features discussed in the previous section. An additional approach to further develop this thesis project is to explore optimization of the compute shader algorithm as discussed in Section 6.3.1.

This project was, as previously mentioned, built using UE v4.26. As of the 26th of May 2021, Unreal Engine 5 (UE5) has been released in early access [50]. This opens up new possibilities of redoing the project using the potential in this new engine. Using the new engine a person could try to experiment with a new type of shader called *mesh shader* [51]. This new type of shader shares features with a compute shader and with any luck it will be better integrated into UE5 than compute shaders are in UE v4.26.

6.4 Reflections

One of the positive effects of this project, is that we can visualize bigger graph structures with limited hardware, since we can better utilize the GPU and put less stress on the CPU to GPU memory bandwidth. This could allow a greater fraction of the population to experiment with information visualization, as well as reduce the environmental damage that buying new hardware entails. Being able to render and interact with big graphs could also increase the understanding of data suitable for being visualized in graph form, such as flight patterns, relationships, *etc.*

References

- [1] R. Mazza, *Introduction to Information Visualization*, 1st ed. London: Springer London, 2009. ISBN 978-1-282-03642-0 [Page 1.]
- [2] O. Kwon, C. Muelder, K. Lee, and K. Ma, “A Study of Layout, Rendering, and Interaction Methods for Immersive Graph Visualization,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 22, no. 7, pp. 1802–1815, Jul. 2016. doi: 10.1109/TVCG.2016.2520921 Conference Name: IEEE Transactions on Visualization and Computer Graphics. [Pages 1, 12, 20, 26, 35, and 44.]
- [3] P. Millais, S. L. Jones, and R. Kelly, “Exploring Data in Virtual Reality: Comparisons with 2D Data Visualizations,” in *Extended Abstracts of the 2018 CHI Conference on Human Factors in Computing Systems*, ser. CHI EA '18. New York, NY, USA: Association for Computing Machinery, Apr. 2018. doi: 10.1145/3170427.3188537. ISBN 978-1-4503-5621-3 pp. 1–6. [Online]. Available: <https://doi.org/10.1145/3170427.3188537> [Page 1.]
- [4] F. Tehnologies, “Guidelines for VR Performance Optimization | Oculus Developers.” [Online]. Available: <https://developer.oculus.com/documentation/native/pc/dg-performance-guidelines/> [Page 1.]
- [5] S. Bryson, “Virtual reality in scientific visualization,” *Communications of the ACM*, vol. 39, no. 5, pp. 62–71, May 1996. doi: 10.1145/229459.229467. [Online]. Available: <https://dl.acm.org/doi/10.1145/229459.229467> [Page 2.]
- [6] E. V. Shikin and A. I. Plis, *Handbook on Splines for the User*. CRC Press, Jul. 1995. ISBN 978-0-8493-9404-1 Google-Books-ID: DL88KouJCQkC. [Pages 2, 12, and 13.]
- [7] O. Zachariadis, A. Teatini, N. Satpute, J. Gómez-Luna, O. Mutlu, O. J. Elle, and J. Olivares, “Accelerating B-spline interpolation on

- GPUs: Application to medical image registration,” *Computer Methods and Programs in Biomedicine*, vol. 193, p. 105431, Sep. 2020. doi: 10.1016/j.cmpb.2020.105431. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S0169260719319601> [Page 2.]
- [8] R. Vuduc and J. Choi, “A Brief History and Introduction to GPGPU,” in *Modern Accelerator Technologies for Geographic Information Science*, X. Shi, V. Kindratenko, and C. Yang, Eds. Boston, MA: Springer US, 2013, pp. 9–23. ISBN 978-1-4614-8745-6. [Online]. Available: https://doi.org/10.1007/978-1-4614-8745-6_2 [Pages 2, 15, 17, and 18.]
- [9] Epic Games, Inc., “Unreal Engine | The most powerful real-time 3D creation platform.” [Online]. Available: <https://www.unrealengine.com/en-US/> [Pages 2, 10, 20, 21, 35, and 43.]
- [10] J. D. Owens, M. Houston, D. Luebke, S. Green, J. E. Stone, and J. C. Phillips, “GPU Computing,” *Proceedings of the IEEE*, vol. 96, no. 5, pp. 879–899, May 2008. doi: 10.1109/JPROC.2008.917757 Conference Name: Proceedings of the IEEE. [Page 2.]
- [11] “Visualiseringscenter C.” [Online]. Available: <http://visualiseringscenter.se/> [Pages 2 and 26.]
- [12] T. Möller, *Real-time rendering*, 4th ed. Boca Raton ; London ; New York: CRC Press, Taylor & Francis Group, 2018. ISBN 978-0-429-22540-6 [Pages 4, 5, 7, 8, 9, 10, 11, 12, and 13.]
- [13] Epic Games, Inc., “EpicGames/UnrealEngine/SourceCode.” [Online]. Available: <https://github.com/EpicGames/UnrealEngine/blob/a27ad66f0a075f3b74ef8f68a4b2b0da4882425e/Engine/Source/Runtime/Core/Public/Math/UnrealMathUtility.h> [Page 4.]
- [14] M. Schirski, T. Kuhlen, M. Hopp, P. Adomeit, S. Pischinger, and C. Bischof, “Efficient visualization of large amounts of particle trajectories in virtual environments using virtual tubelets,” in *Proceedings of the 2004 ACM SIGGRAPH international conference on Virtual Reality continuum and its applications in industry*, ser. VRCAI '04. New York, NY, USA: Association for Computing Machinery, Jun. 2004. doi: 10.1145/1044588.1044615. ISBN 978-1-58113-884-9 pp. 141–147. [Online]. Available: <https://doi.org/10.1145/1044588.1044615> [Pages 4, 12, 14, and 22.]

- [15] B. Russig, M. Salm, and S. Gumhold, “GPU-based Raycasting of Hermite Spline Tubes,” in *2020 IEEE Visualization Conference (VIS)*, Oct. 2020. doi: 10.1109/VIS47514.2020.00012 pp. 26–30. [Pages 4, 11, 14, 23, 24, 26, and 27.]
- [16] M. Kanzler, M. Rautenhaus, and R. Westermann, “A Voxel-Based Rendering Pipeline for Large 3D Line Sets,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 25, no. 7, pp. 2378–2391, Jul. 2019. doi: 10.1109/TVCG.2018.2834372 Conference Name: IEEE Transactions on Visualization and Computer Graphics. [Pages 4, 14, 24, 26, 27, and 35.]
- [17] G. Fabian, C. Massimiliano, P. Sumanta, and D. B. Marco, *Introduction to Computer Graphics*. Chapman and Hall/CRC, Oct. 2014. [Online]. Available: <https://learning.oreilly.com/library/view/introduction-to-computer/9781439852798/> [Pages 8 and 11.]
- [18] NVIDIA Corporation, “DirectX 12 Ultimate Technologies | NVIDIA.” [Online]. Available: <https://www.nvidia.com/sv-se/geforce/technologies/dx12/> [Page 9.]
- [19] Unity Technologies, “Unity Real-Time Development Platform | 3D, 2D VR & AR Engine.” [Online]. Available: <https://unity.com/> [Page 10.]
- [20] T. Tanaka and N. Sakamoto, “Stylized Semi-transparent Streamlines by Stochastic Rendering Approach,” in *2017 Nicograph International (NicoInt)*, Jun. 2017. doi: 10.1109/NICOInt.2017.30 pp. 96–96. [Page 12.]
- [21] D. F. Rogers, *An Introduction to NURBS: With Historical Perspective*, ser. The Morgan Kaufmann Series in Computer Graphics. San Francisco: Elsevier Science & Technology, Morgan Kaufmann, 2000. ISBN 978-1-55860-669-2 [Page 12.]
- [22] Desmos, Inc., “Desmos Bezier Curves.” [Online]. Available: <https://www.desmos.com/calculator/cahqdxeshd?lang=sv-SE> [Page 13.]
- [23] D. H. Ballard and C. M. Brown, *Computer Vision*. Prentice-Hall, 1982. ISBN 978-0-13-165316-0 Google-Books-ID: EfrRAAAA-MAAJ. [Page 13.]
- [24] E. Risser, *True Impostors*. Addison-Wesley Professional, Aug. 2007, ch. 21. ISBN 978-0321515261. [Online].

- Available: <https://developer.nvidia.com/gpugems/gpugems3/part-iv-image-effects/chapter-21-true-impostors> [Page 14.]
- [25] NVIDIA Corporation, “About CUDA,” Mar. 2014. [Online]. Available: <https://developer.nvidia.com/about-cuda> [Page 15.]
- [26] “Compute Shader - OpenGL Wiki.” [Online]. Available: https://www.khronos.org/opengl/wiki/Compute_Shader [Page 17.]
- [27] M. K. Elden, “Implementation and initial assessment of VR for scientific visualisation: Extending Unreal Engine 4 to visualise scientific data on the HTC Vive,” Master’s thesis, University of Oslo, Oslo, Norway, 2017, accepted: 2017-08-08T22:28:03Z. [Online]. Available: <https://www.duo.uio.no/handle/10852/56883> [Page 20.]
- [28] J. Manjón Prado, “Using Unreal Engine as an engineering tool for traffic simulation and analysis,” Master’s thesis, Universitat Politècnica de Catalunya, Barcelona, Spain, May 2020, accepted: 2020-05-13T08:29:36Z Publisher: Universitat Politècnica de Catalunya. [Online]. Available: <https://upcommons.upc.edu/handle/2117/187331> [Page 20.]
- [29] N. Raparthy, E. Acosta, A. Liu, and T. McLaughlin, “GPU-based Motion Matching for Crowds in the Unreal Engine,” in *SIGGRAPH Asia 2020 Posters*, ser. SA ’20 Posters. New York, NY, USA: Association for Computing Machinery, Dec. 2020. doi: 10.1145/3415264.3425474. ISBN 978-1-4503-8113-0 pp. 1–2. [Online]. Available: <https://doi.org/10.1145/3415264.3425474> [Page 20.]
- [30] L. L. Nesi, C. Rorden, and B. C. Munsell, “A Simple and Efficient Cylinder Imposter Approach to Visualize DTI Fiber Tracts,” in *Connectomics in NeuroImaging*, ser. Lecture Notes in Computer Science, G. Wu, P. Laurienti, L. Bonilha, and B. C. Munsell, Eds. Cham: Springer International Publishing, 2017. doi: 10.1007/978-3-319-67159-8_11. ISBN 978-3-319-67159-8 pp. 89–97. [Pages 23, 26, and 27.]
- [31] R. Concheiro, M. Amor, and M. Bóo, “Synthesis of Bézier surfaces on the GPU,” in *Proceedings of the International Conference on Computer Graphics Theory and Applications - Volume 1: GRAPP, (VISIGRAPP 2010)*, vol. 2. SCITEPRESS, May 2010.

- doi: 10.5220/0002847201100115. ISBN 978-989-674-026-9 pp. 110–115. [Online]. Available: <http://www.scitepress.org/Papers/2010/28472> [Page 24.]
- [32] A. Rockwood, K. Heaton, and T. Davis, “Real-time rendering of trimmed surfaces,” in *Proceedings of the 16th annual conference on Computer graphics and interactive techniques*, ser. SIGGRAPH ’89. New York, NY, USA: Association for Computing Machinery, Jul. 1989. doi: 10.1145/74333.74344. ISBN 978-0-89791-312-6 pp. 107–116. [Online]. Available: <https://doi.org/10.1145/74333.74344> [Page 24.]
- [33] R. Concheiro, M. Amor, E. J. Padrón, M. Gil, and X. Martorell, “Rendering of Bézier Surfaces on Handheld Devices,” *Journal of WSCG*, 2013, accepted: 2014-02-03T07:06:53Z Publisher: Václav Skala - Union Agency. [Online]. Available: <http://dspace5.zcu.cz/handle/11025/6983> [Page 24.]
- [34] N. Tatarchuk, “Real-Time Tessellation on GPU,” p. 27, 2007. [Online]. Available: [https://developer.amd.com/wordpress/media/2012/10/Tatarchuk-Tessellation\(EG2007\).pdf](https://developer.amd.com/wordpress/media/2012/10/Tatarchuk-Tessellation(EG2007).pdf) [Page 24.]
- [35] P. Rollin and B. Oster, “OpenGL & CUDA tessellation,” 2011. [Online]. Available: https://www.nvidia.com/content/siggraph/Rollin_Oster_OpenGL_CUDA.pdf [Page 24.]
- [36] G. Nunes, A. Valdetaro, A. Raposo, B. Feijó, and R. d. Toledo, “Rendering Tubes from Discrete Curves Using Hardware Tessellation,” *Journal of Graphics Tools*, vol. 16, no. 3, pp. 123–143, Aug. 2012. doi: 10.1080/2165347X.2012.659610 Publisher: Taylor & Francis _eprint: <https://doi.org/10.1080/2165347X.2012.659610>. [Online]. Available: <https://doi.org/10.1080/2165347X.2012.659610> [Pages 24 and 27.]
- [37] P. Mi, M. Sun, M. Masiane, Y. Cao, and C. North, “AVIST: A GPU-Centric Design for Visual Exploration of Large Multidimensional Datasets,” *Informatics*, vol. 3, no. 4, p. 18, Dec. 2016. doi: 10.3390/informatics3040018 Number: 4 Publisher: Multidisciplinary Digital Publishing Institute. [Online]. Available: <https://www.mdpi.com/2227-9709/3/4/18> [Pages 25 and 26.]
- [38] A. Panagiotidis, G. Reina, M. Burch, T. Pfannkuch, and T. Ertl, “Consistently GPU-Accelerated Graph Visualization,” in *Proceedings of the 8th International Symposium on Visual Information Communication*

- and Interaction*, ser. VINCI '15. New York, NY, USA: Association for Computing Machinery, Aug. 2015. doi: 10.1145/2801040.2801053. ISBN 978-1-4503-3482-2 pp. 35–41. [Online]. Available: <https://doi.org/10.1145/2801040.2801053> [Page 25.]
- [39] P. Mi, M. Sun, M. Masiane, Y. Cao, and C. North, “Interactive Graph Layout of a Million Nodes,” *Informatics*, vol. 3, no. 4, p. 23, Dec. 2016. doi: 10.3390/informatics3040023 Number: 4 Publisher: Multidisciplinary Digital Publishing Institute. [Online]. Available: <https://www.mdpi.com/2227-9709/3/4/23> [Pages 26, 32, and 44.]
- [40] M. Schirski, T. Kuhlen, M. Hopp, P. Adomeit, S. Pischinger, and C. Bischof, “Virtual Tubelets-efficiently visualizing large amounts of particle trajectories,” *Computers and Graphics*, vol. 29, no. 1, pp. 17–27, Feb. 2005. doi: 10.1016/j.cag.2004.11.004. [Online]. Available: <https://doi.org/10.1016/j.cag.2004.11.004> [Pages 26, 27, and 35.]
- [41] Epic Games, Inc., “Profiler Tool Reference.” [Online]. Available: <https://docs.unrealengine.com/4.26/en-US/TestingAndOptimization/PerformanceAndProfiling/Profiler/> [Page 35.]
- [42] Microsoft, “Visual Studio 2019 IDE - Programming Software for Windows.” [Online]. Available: <https://visualstudio.microsoft.com/vs/> [Page 35.]
- [43] C. Conway, “Runtime Mesh Docs • Builtins.” [Online]. Available: <https://runtimesh.koderz.io/builtins.html> [Pages 35 and 44.]
- [44] Intel, Inc., “Intel® Core™ i7-8750H Processor (9M Cache, up to 4.10 GHz) Product Specifications.” [Online]. Available: <https://ark.intel.com/content/www/us/en/ark/products/134906/intel-core-i7-8750h-processor-9m-cache-up-to-4-10-ghz.html> [Page 35.]
- [45] NVIDIA Corporation, “GeForce GTX 1050 Graphics Cards from NVIDIA GeForce.” [Online]. Available: <https://www.nvidia.com/en-in/geforce/products/10series/geforce-gtx-1050/> [Page 35.]
- [46] Dell Computer, “Dell_xps.” [Online]. Available: <https://www.dell.com/sv-se/shop/bärbara-dell-datorer/sf/xps-laptops> [Page 35.]
- [47] S. White, D. Coulter, M. Jacobs, and M. Satran, “High-level shader language (HLSL) - Win32 apps,” posting in Microsoft Windows

- Developers, 2021. [Online]. Available: <https://docs.microsoft.com/en-us/windows/win32/direct3dhls/dx-graphics-hls> [Page 38.]
- [48] Epic Games, Inc., “Stat Commands.” [Online]. Available: <https://docs.unrealengine.com/4.26/en-US/TestingAndOptimization/PerformanceAndProfiling/StatCommands/> [Page 57.]
- [49] NVIDIA Corporation, “NVIDIA Nsight Graphics,” Feb. 2018. [Online]. Available: <https://developer.nvidia.com/nsight-graphics> [Page 60.]
- [50] Epic Games, Inc., “Unreal Engine 5 is now available in Early Access! - Unreal Engine.” [Online]. Available: <https://www.unrealengine.com/en-US/blog/unreal-engine-5-is-now-available-in-early-access> [Page 65.]
- [51] C. Kubisch, “Introduction to Turing Mesh Shaders,” Sep. 2018. [Online]. Available: <https://developer.nvidia.com/blog/introduction-turing-mesh-shaders/> [Page 65.]

Appendix A

Source code

A.1 Compute Shader

```

1 #include "Engine/Private/Common.ush"
2 #include "Engine/Public/Platform.ush"
3
4 uint PackFloat3NormalToUINT(float3 Normal, uint W)
5 {
6     uint X = clamp((sign(Normal.x)*(sign(Normal.x) - 1))
7         *127.5 + Normal.x*127, 0, 255);
8     uint Y = clamp((sign(Normal.y)*(sign(Normal.y) - 1))
9         *127.5 + Normal.y*127, 0, 255);
10    uint Z = clamp((sign(Normal.z)*(sign(Normal.z) - 1))
11        *127.5 + Normal.z*127, 0, 255);
12
13    // Reading from right to left: We want X to be the right-
14    // most 8-bits and W the left-most bits (a.k.a. lower-
15    // order bits and higher order bits)
16    return (W << 24) | (Z << 16) | (Y << 8) | X;
17 }
18
19 // uint8 => offset 8 bits
20 uint PackFloat4ColorToUINT(float4 Color)
21 {
22     const uint R = clamp(Color.r * 255, 0, 255);
23     const uint G = clamp(Color.g * 255, 0, 255);
24     const uint B = clamp(Color.b * 255, 0, 255);
25     const uint A = clamp(Color.a * 255, 0, 255);
26
27     /*
28     // If we look inside Color.h we see that we have:
29     ...

```



```

25         // Win32 x86
26         union { struct{ uint8 B,G,R,A; }; uint32
                AlignmentDummy; };
27         ...
28         In other words: A is the last part and should be
                packed first. B has the first bits and should be
                packed last.
29     */
30     return (A << 24) | (R << 16) | (G << 8) | B;
31 }
32
33 // vector <double, 4> myVector = { 0.2, 0.3, 0.4, 0.5 };
34
35 // ===== Data types and constants
    =====
36 struct MyDynamicMeshVertex
37 {
38     float3 Position;
39     uint Color;
40     float2 TextureCoordinate[8]; // TODO: Check if something
        will happen if this is not the same size as on the CPU
41     uint TangentX;
42     uint TangentZ;
43 };
44
45 // struct PositionBufferOutput
46 //{
47 //     float3 Position;
48 //};
49
50 // ===== Buffers =====
51 // TODO: Check read/write for SRV vs UAV
52 // StructuredBuffer<Line> DataBufferInput;
53
54 // Data buffers
55 StructuredBuffer<float3> DataBufferInput;
56 RWStructuredBuffer<MyDynamicMeshVertex> DataBufferOutput;
57
58 // Index buffers
59 StructuredBuffer<uint> IndexBufferInput;
60 RWStructuredBuffer<uint> IndexBufferOutput;
61
62 // ===== Input for compute shader
    =====
63 //             SV_GroupThreadID = The local 3D thread index
        in a group ([0, numthreads])

```

```

64 //          SV_GroupIndex = The local ID thread index in
        a group (BlockThreadIndex)
65 //          SV_GroupID = The group index for a dispatch (
        BlockIndex)
66 //          SV_DispatchThreadID = The global thread index
        for a dispatch (GroupID(x,y,z)*GroupDim + GroupThreadID
        or GroupID(x,y,z)*numthreadsSize + BlockThreadIndex)
67 // TODO: Consider using SV_GroupIndex as ID thread index as
        BlockThreadIndex. It is not needed for ID structures such
        as (64,1,1) but can be handy for other scenarios.
68 // =====
        =====
69
70 #ifndef NUM_THREADS
71 #define NUM_THREADS 256
72 #endif
73
74 [numthreads(NUM_THREADS, 1, 1)]
75 void Main(uint3 GlobalThreadID : SV_DispatchThreadID, uint3
        GroupThreadID : SV_GroupThreadID, uint3 GroupID :
        SV_GroupID)
76 {
77     const uint nPolygonEdges = ComputeShaderConstants.
        nPolygonEdges;
78     const uint maxCentroidsInGroup = NUM_THREADS /
        nPolygonEdges; // 8
79     const uint nCentroids = ComputeShaderConstants.
        nCentroids;
80
81     const uint currentGroupCentroidIdx = GroupThreadID.x
        / nPolygonEdges;
82     const uint currentCentroidEdge = GroupThreadID.x %
        nPolygonEdges;
83
84     // TODO: For some reason, comparison between int and
        uint with implicit casting stopped working...
85     const uint linearGroupIndex = (ComputeShaderConstants
        .GridSizeX*GroupID.y + GroupID.x)
86     * maxCentroidsInGroup + currentGroupCentroidIdx;
87
88
89     if(currentGroupCentroidIdx < maxCentroidsInGroup &&
        linearGroupIndex < nCentroids) // Only let
        nPolygonEdges number of threads in each Group
        enter the shader. Also, only let nCentroids thread
        execute the shader
90     {

```

```

91     float3 centroidVertex = DataBufferInput[
        linearGroupIndex]; // An FVector has 12 bytes, the
        same as float3 (without padding, apparently)
92
93     const float epsilon = 1.19209290e-007;
94     const float angularStepSizeRadians =
        ComputeShaderConstants.AngularStepSizeRadians;
95     const int nPolygonEdges = ComputeShaderConstants.
        nPolygonEdges;
96     const float radius = ComputeShaderConstants.Radius;
97     // TODO: Do we use the time for anything? Can
        be used later to update positions from a
        server
98     float time = ComputeShaderVariables.TotalTime;
99     const float thetaThread = 2 * PI -
        currentCentroidEdge * angularStepSizeRadians; //
        assign an angle for each thread
100
101     const int lineSegmentVerticesCount = nPolygonEdges;
        // The number of vertices for each parameterized
        circle
102
103     const int vertexBufferThreadOffset =
        lineSegmentVerticesCount * linearGroupIndex +
        currentCentroidEdge; // Jump every two polygons/
        lines OR: Jump the number of vertices for each
        line
104     const int indexBufferThreadOffset = 6 *
        linearGroupIndex * nPolygonEdges; // 3 vertices *
        2 triangles * polyEdgeCount = 6*polyEdgeCount
        indices per line
105
106     float3 segmentTangentVector, tempVector;
107
108     // the biSegmentTangentVector is the vector in
        between the current tangent vector and the next
        tangent vector
109     float3 biSegmentTangentVector;
110     float biSegmentAngle;
111     float majorAxis = radius;
112     const float thresholdAngle = PI / 2.0f;
113
114     // TODO: This assumes that there are at least
        2 centroids or at least one line in the
        mesh component
115     if (linearGroupIndex == 0)
116     {

```

```

117         // Case 1: First centroid – will use
           // the first and second centroid to
           // determine segment tangent vector
118         segmentTangentVector = normalize(DataBufferInput
           [1] - centroidVertex);
119         biSegmentTangentVector = segmentTangentVector;
120     }
121     else
122     {
123         // For all other centroids the tangent
           // vector is:  $v_n - v_{n-1}$ 
124         segmentTangentVector = normalize(centroidVertex -
           DataBufferInput[linearGroupIndex - 1]);
125         if(linearGroupIndex != nCentroids - 1)
126         {
127             tempVector = normalize((DataBufferInput[
           linearGroupIndex + 1] - centroidVertex));
128             biSegmentTangentVector = normalize(tempVector
           - segmentTangentVector);
129         }
130         else
131         {
132             biSegmentTangentVector = segmentTangentVector
           ;
133         }
134     }
135
136     biSegmentAngle = acos(dot(-biSegmentTangentVector ,
           segmentTangentVector));
137
138     float3 B;
139     float3 C;
140     // Given a normal  $N$  (i.e.
           // segmentTangentVector), this function
           // computes two (unit length) basis vectors  $B$ 
           // and  $C$  for the plane  $N$ . See Random.ush
           // included from /Engine/Private/Common.ush.
141     //  $B$  starts as UpVector (0,0,1) or
           // ForwardVector (1,0,0) but is overwritten.
           // Probably to save space (we don't actually
           // need to store the UpVector explicitly).
142
143     if (biSegmentAngle < thresholdAngle && biSegmentAngle
           > -thresholdAngle)
144     {
145         B = normalize(cross(biSegmentTangentVector ,
           segmentTangentVector));

```

```

146         C = biSegmentTangentVector;
147         majorAxis = radius / sin(biSegmentAngle);
148     }
149     else
150     {
151         FindBestAxisVectors(segmentTangentVector, B, C);
152         majorAxis = radius;
153     }
154
155     //Only store the exterior shell around a
156     centroid:
157     float3 newVertice = centroidVertex.xyz + B.xyz *
158         radius * cos(thetaThread) + majorAxis * sin(
159             thetaThread) * C.xyz;
160     float3 normal = normalize(newVertice - centroidVertex
161         .xyz);
162
163     // for debugging purposes
164     DataBufferOutput[vertexBufferThreadOffset].Position =
165         newVertice;
166     DataBufferOutput[vertexBufferThreadOffset].Color =
167         PackFloat4ColorToUINT(colorTest);
168     DataBufferOutput[vertexBufferThreadOffset].TangentX =
169         PackFloat3NormalToUINT(normalize(
170             segmentTangentVector), 1);
171     DataBufferOutput[vertexBufferThreadOffset].TangentZ =
172         PackFloat3NormalToUINT(normalize(normal), 1);
173
174     const int firstStartIndex = linearGroupIndex *
175         lineSegmentVerticesCount;
176     const int closeLoopIndex = firstStartIndex;
177     const int lastStartIndex = firstStartIndex + (
178         nPolygonEdges - 1);
179
180     const int firstEndIndex = lastStartIndex + 1;
181     const int lastEndIndex = firstEndIndex + (
182         nPolygonEdges - 1);
183
184     uint stride = 0;
185
186     if (linearGroupIndex != nCentroids - 1) // We only
187         want to close the mesh excluding the last centroid
188         . The last centroid is not connected to anything/
189         closed end.
190     {
191         // Write triangles. Does it quad-wise, i.e.

```

```

    each thread writes a quad (two triangles)
    => one thread per side
178     if (currentCentroidEdge > 0)
179     {
180         const int currentIndex = lastStartIndex -
            currentCentroidEdge;
181         const int indexQuadOffset = 6 *
            currentCentroidEdge;

182         IndexBufferOutput[indexBufferThreadOffset +
            indexQuadOffset + stride] = currentIndex;
183         stride = stride + 1;
184         IndexBufferOutput[indexBufferThreadOffset +
            indexQuadOffset + stride] = nPolygonEdges
            + currentIndex;
185         stride = stride + 1;
186         IndexBufferOutput[indexBufferThreadOffset +
            indexQuadOffset + stride] = currentIndex +
            1;
187         stride = stride + 1;
188
189         IndexBufferOutput[indexBufferThreadOffset +
            indexQuadOffset + stride] = currentIndex +
            1;
190         stride = stride + 1;
191         IndexBufferOutput[indexBufferThreadOffset +
            indexQuadOffset + stride] = nPolygonEdges
            + currentIndex;
192         stride = stride + 1;
193         IndexBufferOutput[indexBufferThreadOffset +
            indexQuadOffset + stride] = nPolygonEdges
            + currentIndex + 1;
194         stride = stride + 1;
195     }
196
197     // Wrap/tie the mesh, special case when
    we get around
199     else if (currentCentroidEdge == 0)
200     {
201         IndexBufferOutput[indexBufferThreadOffset +
            stride] = closeLoopIndex;
202         stride = stride + 1;
203         IndexBufferOutput[indexBufferThreadOffset +
            stride] = lastStartIndex;
204         stride = stride + 1;
205         IndexBufferOutput[indexBufferThreadOffset +
            stride] = lastEndIndex;

```

```

206         stride = stride + 1;
207
208         IndexBufferOutput[indexBufferThreadOffset +
            stride] = closeLoopIndex;
209         stride = stride + 1;
210         IndexBufferOutput[indexBufferThreadOffset +
            stride] = lastEndIndex;
211         stride = stride + 1;
212         IndexBufferOutput[indexBufferThreadOffset +
            stride] = firstEndIndex;
213         stride = stride + 1;
214     }
215 }
216 }
217
218
219 }

```

A.2 Sequential Implementation

```

1      // Used for triangle indexing
2      int Idx, NextIdx, Counter;
3
4      // Angle at which the vertexes are generated around
5      each centroid
6      float Angle, OffsetAngle;
7
8      // Angle and tangent between two segments
9      float BiSegmentAngle, ThresholdAngle;
10     FVector BiSegmentTangent;
11
12     // Major and minor axis lengths for ellipsis
13     calculations
14     float MinorAxis, MajorAxis;
15
16     // Temp values for positions of the vertexes
17     FVector CenterVertex, NewVertex;
18     FVector A, B, TempA;
19     FVector TangentX, TangentY, TangentZ, TangentTemp;
20     FVector ReferencePlaneNormal = { 1, 0, 0 };
21     FVector GlobalUp = { 0, 0, 1 };
22
23     // get spline parameters from meshcomponent
24     int NumPolygons = MyMeshComponent->NumPolygons;
25     float AngleStepRadians = 2 * PI / NumPolygons;

```

```

24     SplineRadius = MyMeshComponent->Radius;
25     int NumCentroids = Centroids->Num();
26
27     MinorAxis = SplineRadius;
28     ThresholdAngle = PI / 2.0f;
29
30     Positions.AddUninitialized(NumCentroids * NumPolygons
31                               );
32     Colors.AddUninitialized(NumCentroids * NumPolygons);
33     Normals.AddUninitialized(NumCentroids * NumPolygons);
34     Tangents.AddUninitialized(NumCentroids * NumPolygons)
35     ;
36     Triangles.AddUninitialized(NumCentroids * NumPolygons
37                               * 6);
38     TArray<FVector2D> EmptyTexCoords;
39     TArray<FRuntimeMeshTangent> EmptyTangents;
40
41     for (int32 i = 0; i < NumCentroids; i++)
42     {
43         CenterVertex = (*Centroids)[i];
44         MajorAxis = MinorAxis;
45
46         // first centroid
47         if (i == 0)
48         {
49             // TangentX is the vector between
50             current and next centroid
51             TangentX = (*Centroids)[1] -
52                       CenterVertex;
53             TangentX.Normalize();
54             TangentX.FindBestAxisVectors(A,
55                                           BiSegmentTangent);
56
57             // DrawDebugLine(GetWorld(),
58             CenterVertex, CenterVertex + A * 400,
59             FColor::White, true, -1.0f, 0, 10.0f);
60         }
61         else if (i != NumCentroids - 1)
62         {
63             // TangentTemp stores the previous
64             TangentX
65             TangentTemp = TangentX;
66             TangentX = (*Centroids)[i + 1] -
67                       CenterVertex;
68             TangentX.Normalize();
69         }
70     }

```



```

60          // Calc angle between two segment's
           tangent vectors
61      BiSegmentAngle = acos(FVector::
           DotProduct(-TangentTemp, TangentX)
           );
62
63      // if the angle between current and
           last tangent is bellow a certain
           threshold
64      if (BiSegmentAngle < ThresholdAngle
           && BiSegmentAngle > -
           ThresholdAngle)
65      {
66          // Major axis of ellipsis
           defined by the angle at
           which the cylinder is cut
67          MajorAxis = MinorAxis / sin(
           BiSegmentAngle / 2.0f);
68
69          // The BiSegmenTangent is the
           tangent in between
           current and last segments'
           s tangents
70          BiSegmentTangent = TangentX -
           TangentTemp;
71          BiSegmentTangent.Normalize();
72
73          // Save previous A vector in
           TangentTemp
74          TangentTemp = A;
75          A = FVector::CrossProduct(
           TangentX, BiSegmentTangent
           );
76          A.Normalize();
77      }
78      else
79      {
80          TangentTemp = A;
81          TangentX.FindBestAxisVectors(
           A, BiSegmentTangent);
82      }
83  }
84  // last centroid
85  else
86  {
87      TangentTemp = A;
88      TangentX.FindBestAxisVectors(A,

```

```

BiSegmentTangent);
89     }
90
91     // Projecting A on the reference plane
92     TempA = A - (FVector::DotProduct(A,
          ReferencePlaneNormal)) *
          ReferencePlaneNormal;
93
94     if(TempA.Size() == 0)
95     {
96         OffsetAngle = 0;
97     }
98     else
99     {
100         TempA.Normalize();
101         if (FVector::DotProduct(
          ReferencePlaneNormal, FVector::
          CrossProduct(TempA, GlobalUp)) >
          0)
102         {
103             OffsetAngle = acos(FVector::
          DotProduct(TempA, GlobalUp
          ));
104         }
105         else
106         {
107             OffsetAngle = -acos(FVector::
          DotProduct(TempA, GlobalUp
          ));
108             //DrawDebugLine(GetWorld(),
          CenterVertex, CenterVertex
          + TempA * 300, FColor::
          Red, true, -1.0f, 0, 10.0f
          );
109         }
110     }
111
112     for (int j = 0; j < NumPolygons; j++)
113     {
114         // The angle offset makes sure there
          are no twisting of the tube from
          segment to segment
115         Angle = AngleStepRadians * j +
          OffsetAngle;
116
117         // The new vertex is created on the
          circumfrance of an ellipse

```

```

118         NewVertex = CenterVertex + MinorAxis
            * cos(Angle) * A + MajorAxis * sin
            (Angle) * BiSegmentTangent;
119
120         // TangentY is calculated as the
            Normal vector
121         TangentY = NewVertex - CenterVertex;
122         TangentY.Normalize();
123
124         // Idx for Vertice Color and normal
            components
125         Idx = i * NumPolygons + j;
126         Positions[Idx] = NewVertex;
127         Colors[Idx] = FColor::Green;
128         Normals[Idx] = TangentY;
129     }
130 }
131
132 // add triangle indices
133 Counter = 0;
134 for (int i = 0; i < NumCentroids - 1; i++)
135 {
136     // one quad for each polygon in the tube
137     for (int j = 0; j < NumPolygons; j++)
138     {
139         Idx = i * NumPolygons + j;
140         NextIdx = ((Idx + 1) % NumPolygons)
            == 0 ? NumPolygons * i : Idx + 1;
141
142         Triangles[Counter] = Idx;
143         Triangles[Counter + 1] = NextIdx;
144         Triangles[Counter + 2] = Idx +
            NumPolygons;
145
146         Triangles[Counter + 3] = Idx +
            NumPolygons;
147         Triangles[Counter + 4] = NextIdx;
148         Triangles[Counter + 5] = NextIdx +
            NumPolygons;
149         Counter += 6;
150     }
151 }
152 }
153 UpdateSectionFromComponents(0, 0, Positions,
    Triangles, Normals, EmptyTexCoords, Colors,
    EmptyTangents);
154 return;

```

155 }

TRITA-EECS-EX-2021:500